Department of Information Technology

Faculty of Engineering Sciences and Technology

Hamdard University Karachi

Combined Laboratory Manual

Scripting Programs

Fall 2026 Revised Edition

Part I: DFSP-5208: Scripting Programs

BS Digital Forensics and Cyber Security

Part II: CCSP-5213: Scripting Program and Cloud Architecture

BS Cloud Applications Development and Operations

Instructor: Iqbal Uddin Khan

Email: Iqbaluddin.khan@hamdard.edu.pk

Academic Year 2026 | Fall 2026

All tools, platforms, and software specified in this manual are free and open source. Version numbers are minimums rather than claims about the newest release, and tool and license statements were re-verified for Fall 2026.

Table of Contents

Table of Contents **2**
Software & Tools Reference (Free / Open-Source Stack) **8**
How to Use This Manual **10**
Windows students: use WSL2 with systemd 10
Evidence rules that apply to every lab 10
Lab-to-lecture-week mapping 11
Part I: DFSP-5208: Scripting Programs **12**
Lab 1: Introduction to Python Scripting: Writing and Executing Basic Scripts **13**
Title 13
Objectives 13
Preamble / Short Theory 13
Prerequisites 13
Step-by-Step Guide 13
Required Outcomes / Tasks 16
Lab 2: File Handling and Automation in Python **17**
Title 17
Objectives 17
Preamble / Short Theory 17
Prerequisites 17
Step-by-Step Guide 17
Required Outcomes / Tasks 19
Lab 3: Command-Line Tools with Python (argparse) **20**
Title 20
Objectives 20
Preamble / Short Theory 20
Prerequisites 20
Step-by-Step Guide 20
Required Outcomes / Tasks 23
Lab 4: Fundamentals of Bash Scripting **24**
Title 24
Objectives 24
Preamble / Short Theory 24
Prerequisites 24
Step-by-Step Guide 24
Required Outcomes / Tasks 26
Lab 5: Automating System Tasks with Bash (cron) **27**
Title 27
Objectives 27
Preamble / Short Theory 27
Prerequisites 27
Step-by-Step Guide 27
Required Outcomes / Tasks 29
Lab 6: Web Scraping with Python (BeautifulSoup) **30**
Title 30

Objectives 30
Preamble / Short Theory 30
Prerequisites 30
Step-by-Step Guide 30
Required Outcomes / Tasks 32
Lab 7: Node.js Scripting for File System Automation 33
Title 33
Objectives 33
Preamble / Short Theory 33
Prerequisites 33
Step-by-Step Guide 33
Required Outcomes / Tasks 35
Lab 8: Deploying Node.js Scripts as Serverless Functions 36
Title 36
Objectives 36
Preamble / Short Theory 36
Prerequisites 36
Step-by-Step Guide 36
Required Outcomes / Tasks 38
Lab 9: REST API Interaction: Fetching and Parsing JSON Data 39
Title 39
Objectives 39
Preamble / Short Theory 39
Prerequisites 39
Step-by-Step Guide 39
Required Outcomes / Tasks 43
Lab 10: Automating Security Workflows with APIs 45
Title 45
Objectives 45
Preamble / Short Theory 45
Prerequisites 45
Step-by-Step Guide 45
Required Outcomes / Tasks 48
Lab 11: Error Handling and Debugging in Scripts 49
Title 49
Objectives 49
Preamble / Short Theory 49
Prerequisites 49
Step-by-Step Guide 49
Required Outcomes / Tasks 53
Lab 12: Logging and Monitoring in Security Scripts 54
Title 54
Objectives 54
Preamble / Short Theory 54
Prerequisites 54
Step-by-Step Guide 54

Required Outcomes / Tasks....56
Lab 13: Introduction to Docker: Containerizing Forensic Tools....58
Title....58
Objectives....58
Preamble / Short Theory....58
Prerequisites....58
Step-by-Step Guide....58
Required Outcomes / Tasks....60
Lab 14: Automating Container Deployment....61
Title....61
Objectives....61
Preamble / Short Theory....61
Prerequisites....61
Step-by-Step Guide....61
Required Outcomes / Tasks....62
Lab 15: CI/CD Pipeline Automation for Secure Deployment....63
Title....63
Objectives....63
Preamble / Short Theory....63
Prerequisites....63
Step-by-Step Guide....63
Required Outcomes / Tasks....66
Lab 16: Capstone Lab: Designing and Deploying a Complete Forensic Pipeline....67
Title....67
Objectives....67
Preamble / Short Theory....67
Prerequisites....67
Step-by-Step Guide....67
Required Outcomes / Tasks....71
Part II: CCSP-5213: Scripting Program and Cloud Architecture....73
Lab 1: Introduction to Python Scripting: Writing and Executing Basic Scripts....74
Title....74
Objectives....74
Preamble / Short Theory....74
Prerequisites....74
Step-by-Step Guide....74
Required Outcomes / Tasks....77
Lab 2: File Handling and Automation in Python....78
Title....78
Objectives....78
Preamble / Short Theory....78
Prerequisites....78
Step-by-Step Guide....78
Required Outcomes / Tasks....80
Lab 3: Command-Line Tools with Python (argparse)....82
Title....82

Objectives 82
Preamble / Short Theory 82
Prerequisites 82
Step-by-Step Guide 82
Required Outcomes / Tasks 85
Lab 4: Fundamentals of Bash Scripting 86
Title 86
Objectives 86
Preamble / Short Theory 86
Prerequisites 86
Step-by-Step Guide 86
Required Outcomes / Tasks 88
Lab 5: Automating System Tasks with Bash (cron) 89
Title 89
Objectives 89
Preamble / Short Theory 89
Prerequisites 89
Step-by-Step Guide 89
Required Outcomes / Tasks 91
Lab 6: Web Scraping with Python (BeautifulSoup) 92
Title 92
Objectives 92
Preamble / Short Theory 92
Prerequisites 92
Step-by-Step Guide 92
Required Outcomes / Tasks 94
Lab 7: Node.js Scripting for File System Automation 95
Title 95
Objectives 95
Preamble / Short Theory 95
Prerequisites 95
Step-by-Step Guide 95
Required Outcomes / Tasks 97
Lab 8: Deploying Node.js Scripts as Serverless Functions 98
Title 98
Objectives 98
Preamble / Short Theory 98
Prerequisites 98
Step-by-Step Guide 98
Required Outcomes / Tasks 101
Lab 9: REST API Interaction: Fetching and Parsing JSON Data 102
Title 102
Objectives 102
Preamble / Short Theory 102
Prerequisites 102
Step-by-Step Guide 102

Required Outcomes / Tasks 107
Lab 10: Automating Workflows with APIs 108
Title 108
Objectives 108
Preamble / Short Theory 108
Prerequisites 108
Step-by-Step Guide 108
Required Outcomes / Tasks 111
Lab 11: Error Handling and Debugging in Scripts 112
Title 112
Objectives 112
Preamble / Short Theory 112
Prerequisites 112
Step-by-Step Guide 112
Required Outcomes / Tasks 116
Lab 12: Logging and Monitoring in Cloud-Based Scripts 117
Title 117
Objectives 117
Preamble / Short Theory 117
Prerequisites 117
Step-by-Step Guide 117
Required Outcomes / Tasks 120
Lab 13: Introduction to Docker: Containerizing Applications 121
Title 121
Objectives 121
Preamble / Short Theory 121
Prerequisites 121
Step-by-Step Guide 121
Required Outcomes / Tasks 123
Lab 14: Automating Container Deployment 124
Title 124
Objectives 124
Preamble / Short Theory 124
Prerequisites 124
Step-by-Step Guide 124
Required Outcomes / Tasks 125
Lab 15: CI/CD Pipeline Automation 127
Title 127
Objectives 127
Preamble / Short Theory 127
Prerequisites 127
Step-by-Step Guide 127
Required Outcomes / Tasks 130
Lab 16: Capstone Lab: Designing and Deploying a Complete Cloud-Native Application 131
Title 131
Objectives 131

Preamble / Short Theory 131
Prerequisites 131
Step-by-Step Guide 131
Required Outcomes / Tasks 136
Change Log: Fall 2026 Revision 137
Changes made for each finding 137
New material added 139
Formatting standard applied 140
Verification status and open items 140

Software & Tools Reference (Free / Open-Source Stack)

The table below lists every tool, platform, and library used across both courses in this manual, together with its License type and the labs that depend on it. Wherever the original course outline referenced a paid or closed-source service (for example AWS Lambda, Azure Functions or Docker Hub private repositories), an equivalent free and open-source substitute is used so that every exercise can be completed at zero cost, on a student's own machine.

Category	Tool / Platform	License / Cost	Used In
Language runtime	Python 3.12 or newer (3.13 and 3.14 also work)	PSF License - free, open source	Labs 1–3, 6, 9–16
Language runtime	Node.js current LTS (22 or 24)	MIT - free, open source (OpenJS Foundation)	Labs 7–16
Editor	Visual Studio Code (or VSCodium, fully OSS)	Free to use / MIT (VSCodium)	All labs
Shell	GNU Bash 5.x (native Linux/macOS; WSL2 with systemd on Windows)	GPL - free, open source	Labs 4–5, 13–16
Scheduler	cron	Bundled with Linux — free, open source	Labs 5, 10
Web scraping	requests + BeautifulSoup4	Apache-2.0 / MIT - free, open source	Lab 6
Version control	Git + GitHub (free tier) or Gitea (fully OSS, self-hosted)	GPL (Git) - free	Labs 1, 3, 7, 11, 15–16
Containers	Docker Engine CE (or Podman, fully OSS), installed once in Lab 1	Apache-2.0 - free, open source	Labs 8 (optional), 10, 13–16
Serverless / FaaS	Local test harness (required); OpenFaaS + faasd CE (optional)	Harness: course material. faasd CE: non-commercial, personal use under the faasd CE EULA (confirm for classroom use)	Lab 8
Database	SQLite and the sqlite3 command-line client	Public domain - free	Labs 9–10
Mail testing	Mailpit (or Python aiosmtpd)	MIT / Apache-2.0 - free, open source	Labs 10, 16
Debugger	VS Code built-in debugger and Python pdb	Free (bundled)	Lab 11
Logging	Python logging, Winston (Node.js), lnav	Stdlib / MIT / BSD - free, open source	Lab 12
Container registry	Docker Registry (registry:2)	Apache-2.0 - free, open source	Labs 8 (optional), 14–16

Category	Tool / Platform	License / Cost	Used In
Object storage emulator	Moto (S3 emulator) with boto3 and the AWS CLI	Apache-2.0 - free, open source	Labs 5, 9, 14, 16 (CCSP-5213)
CI/CD	GitHub Actions (free tier) and Jenkins (lts-jdk21)	GitHub free tier / MIT (Jenkins)	Labs 15–16
Security scanning	Trivy	Apache-2.0 - free, open source	Lab 15
Diagramming	Excalidraw or draw.io (free tier)	MIT (Excalidraw) - free	Lab 16 (capstone)

Version numbers are minimums, not claims about the newest release. Students should run the official installer or package-manager command for their OS, which resolves to a current supported version. Tool and License statements for faasd, Jenkins and MinIO were re-checked for the Fall 2026 revision and should be re-checked just before term because they can change again.

How to Use This Manual

This manual is organized into two parts, one per course, each containing 16 sequential labs mapped to the corresponding weekly theory schedule:

- Part I: DFSP-5208 (Scripting Programs): Labs 1–16, forensic/cyber-security automation focus.
- Part II: CCSP-5213 (Scripting Program and Cloud Architecture): Labs 1–16, cloud application/DevOps automation focus.

Every lab follows the same six-section structure so students always know where to look:

- **Title** - the lab name and lecture week.
- **Objectives** - what the student will be able to do after completing the lab.
- **Preamble / Short Theory** - the concept(s) underpinning the lab, in plain language.
- **Prerequisites** - exactly what must already be installed or completed before starting.
- **Step-by-Step Guide** - numbered, copy-paste-able commands and code with explanations.
- **Required Outcomes / Tasks** - the specific deliverables the student must submit.

Each lab begins on a new page for easy printing and photocopying. All headings are bookmarked so they can be navigated directly from Word's Navigation Pane (View ▸ Navigation Pane) or linked to from other documents/LMS pages.

Windows students: use WSL2 with systemd

The commands in this manual are Linux commands (source, ~, $(pwd), sudo apt, cron). Windows 11 students must therefore work inside WSL2 with systemd enabled, which cron and faasd need. Open PowerShell as administrator and run the first command below, then inside Ubuntu add the two lines to /etc/wsl.conf and restart WSL from PowerShell.

POWERSHELL

```
wsl --install -d Ubuntu
# inside Ubuntu: sudo nano /etc/wsl.conf and add
#   [boot]
#   systemd=true
# then, in PowerShell:
wsl --shutdown
```

The few commands that differ outside WSL are noted in the labs: activating the virtual environment (.venv\Scripts\Activate.ps1 in PowerShell) and Docker bind mounts (-v "${PWD}/out:/app/out" in PowerShell, without --user).

Evidence rules that apply to every lab

- Every screenshot must show the system clock, and every lab asks for one personal value (for example your roll number), so submissions can be told apart.
- Timestamps are written in UTC, and sample data is generated with a fixed random seed (42) so that results can be checked.
- Required work is graded on the local tools; steps marked optional (for example faasd in Lab 8) are extensions.

Lab-to-lecture-week mapping

Week 9 is the midterm and has no lab, so Labs 9–16 are taught one lecture week later than their numbers suggest. Lab 14 shares Week 14 (second half) or Week 15 with the CI/CD lab.

Manual lab	Lecture week	Topic
Lab 1–8	Weeks 1–8	Python, Bash, cron, scraping, Node.js, serverless
(none)	Week 9	Midterm, no lab
Lab 9	Week 10	REST/JSON and evidence handling
Lab 10	Week 11	Workflow automation and alerts
Lab 11	Week 12	Errors and debugging
Lab 12	Week 13	Logging and monitoring
Lab 13	Week 14	Docker
Lab 14	Week 14 (second half) or 15	Registry and object storage
Lab 15	Week 15	CI/CD and security scanning
Lab 16	Week 16	Capstone

Part I: DFSP-5208: Scripting Programs

Program: BS Digital Forensics and Cyber Security

This part contains 16 sequential laboratory sessions (Lab 1 – Lab 16) aligned to the weekly theory schedule of DFSP-5208. Every lab uses only free and open-source tools and platforms (minimum versions are stated in each lab); no paid cloud accounts, licenses, or subscriptions are required to complete any exercise in this part.

Lab 1: Introduction to Python Scripting: Writing and Executing Basic Scripts

Course: DFSP-5208 | Scripting Programs | Week 1 | Program: BS Digital Forensics and Cyber Security

Title

Introduction to Python Scripting: Writing and Executing Basic Scripts

Objectives

- Install and verify a modern Python 3 toolchain using only free, open-source components.
- Understand how scripting differs from compiled programming and why it matters for forensic automation.
- Write, save, and execute a first Python script from the command line and from VS Code.
- Use a virtual environment to keep project dependencies isolated and reproducible.

Preamble / Short Theory

Scripting languages such as Python are interpreted, high-level languages optimized for quickly automating repetitive tasks rather than building large compiled systems. In a digital forensics lab that must process seized evidence logs, monitor chain-of-custody events, and automate repetitive investigative tasks, analysts rarely have time to write and compile a full application every time a repetitive task appears, instead they write a short script, run it immediately, and iterate. This lab establishes the tool-chain (Python 3, a code editor, and a virtual environment) that will be reused throughout the semester, and introduces the read-write-execute cycle that underlies every later lab.

Prerequisites

- A laptop/desktop running Ubuntu or Kubuntu 24.04 LTS or newer, macOS, or Windows 11 with WSL2 (all free to use; Ubuntu is fully open source). Windows students complete every lab inside WSL2 with systemd enabled (see How to Use This Manual).
- Administrative rights to install software.
- Internet access for the initial download only (all tools work offline afterwards).
- No prior programming experience required.

Step-by-Step Guide

Step 1: Install Python 3.12 or newer (PSF open-source license)

Download the installer from the official Python website (python.org/downloads) or, on Ubuntu, install directly from the repositories. Ubuntu 24.04 provides Python 3.12, which is sufficient for every lab; Python 3.13 or 3.14 is equally fine if your lab PC already has it. Windows/macOS users must tick “Add Python to PATH” during installation.

BASH

```
# Ubuntu / Kubuntu / WSL2
sudo apt update
sudo apt install -y python3 python3-venv python3-pip
python3 --version   # confirm 3.12 or newer
```

Step 2: Install Visual Studio Code (free) or VSCodium (fully open source)

VS Code is free to use; students who prefer a build with no telemetry and a fully open-source license should install VSCodium instead, both use the same editor engine and Python extension. Windows students install VS Code on Windows and add the WSL extension so the editor opens folders inside WSL2.

BASH

```
# Ubuntu (VS Code via snap) - or install the .deb from
code.visualstudio.com
sudo snap install code --classic
# Fully open-source alternative:
# sudo snap install codium --classic
```

Step 3: Create a project folder and virtual environment

A virtual environment keeps this course's Python packages separate from the rest of the system.

BASH

```
mkdir -p ~/forensic-scripting/lab01 && cd ~/forensic-scripting/lab01
python3 -m venv .venv
source .venv/bin/activate        # Windows PowerShell (outside
WSL): .venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
```

Step 4: Write the first script

In VS Code, create a file named hello_forensic.py with the following content. It prints the environment, a UTC timestamp and the SHA-256 hash of the script itself, which introduces the integrity idea used all semester.

PYTHON

```
import hashlib, pathlib, platform, datetime

ROLL = "YOUR-ROLL-NUMBER"   # replace with your own roll number
print("Scripting environment ready.")
print("Student:", ROLL)
print(f"Python {platform.python_version()} running on
{platform.system()}")
print("Course: DFSP-5208 - Scripting Programs")
print("UTC:",
datetime.datetime.now(datetime.timezone.utc).isoformat(timespec="secon
ds"))
print("SHA256 of this script:",
hashlib.sha256(pathlib.Path(__file__).read_bytes()).hexdigest())
```

Step 5: Execute the script from the terminal

Run the script and observe the interpreted, line-by-line execution.

BASH

```
python hello_forensic.py
```

Step 6: Run the same script inside VS Code

Open the folder in VS Code (code .), install the Python extension when prompted, select the .venv interpreter (bottom-right status bar), then use Run ▸ Run Without Debugging.

Step 7: Install Git and Docker once (used from Lab 3 and Lab 8 onward)

Install both tools now so later labs do not have to repeat the set-up. Docker is installed from the Ubuntu repositories, so no unreviewed script is piped into a root shell. Log out and back in (or run newgrp docker) after adding yourself to the docker group.

BASH

```
sudo apt install -y git docker.io
sudo systemctl enable --now docker
sudo usermod -aG docker $USER && newgrp docker
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git --version && docker --version
docker run --rm hello-world
```

Podman (sudo apt install podman) is a fully open-source, daemon-less alternative; Docker's own apt repository (docs.docker.com) can be used instead of docker.io if your instructor prefers the upstream package.

Step 8: Exclude the virtual environment from Git

Create a .gitignore file so that the virtual environment, caches, secrets and evidence files are never committed, then make the first commit.

TEXT

```
.venv/
__pycache__/
.env
*.log
evidence/
```

BASH

```
git init
git add .gitignore hello_forensic.py
git commit -m "Lab 1: first script and .gitignore"
git status    # .venv must not be listed
```

Required Outcomes / Tasks

- Screenshot of `python3 --version` showing 3.12 or newer, with the system clock visible.
- Submit hello_forensic.py (with your own roll number filled in) together with a terminal screenshot of its output, system clock visible.
- One-paragraph written reflection: how does an interpreted script differ from a compiled C/Java program?
- Screenshot of `git status` showing that the virtual environment (.venv) folder is excluded by .gitignore (used from Lab 3 onward).
- Screenshot of `docker run --rm hello-world` succeeding (or a note from your instructor that Docker is not available on your lab PC).

Lab 2: File Handling and Automation in Python

Course: DFSP-5208 | Scripting Programs | Week 2 | Program: BS Digital Forensics and Cyber Security

Title

File Handling and Automation in Python

Objectives

- Read, write, and append text and CSV files using Python's standard library only.
- Automate a repetitive forensic data-processing task without manual copy-pasting.
- Apply context managers (with) to guarantee files are closed safely.

Preamble / Short Theory

Most real forensic automation tasks begin and end with a file: a log dump, a CSV export, or a folder of evidence files that must be renamed or summarized. Python's built-in `open()`, the `csv` module, and the `pathlib` module (all part of the free standard library, no extra installation needed) are sufficient for the overwhelming majority of file-handling tasks and are the foundation for the log-parsing and API labs later in the course.

Programs also fail: a file may be missing or a row malformed. A `try/except` block lets a script report the problem clearly instead of crashing; this lab uses it once, and Lab 11 develops the idea fully.

Prerequisites

- Lab 1 completed (Python 3.12 or newer + venv working).
- A sample CSV file (students may generate one with the script in Step 2 if none is supplied).

Step-by-Step Guide

Step 1: Activate the environment and create a working folder

BASH

```
cd ~/forensic-scripting
mkdir lab02 && cd lab02
source ../lab01/.venv/bin/activate
```

Step 2: Generate a sample dataset

Create generate_sample.py to produce a synthetic forensic evidence dataset for practice (no real/sensitive data is used). The fixed random seed means every student gets the same data, so results can be checked: with seed 42 you should see 12 ERROR, 16 WARN and 22 INFO rows.

PYTHON

```
import csv, random, datetime

random.seed(42)                         # same data on every run, so
answers can be checked
now = datetime.datetime.now(datetime.timezone.utc)
rows = []
for i in range(50):
    ts = now - datetime.timedelta(minutes=i * 7)
    rows.append([ts.isoformat(timespec="seconds"), f"host-
{random.randint(1, 5)}",
                 random.choice(["INFO", "WARN", "ERROR"]), f"event id
{i}"])
with open("sample_data.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerow(["timestamp", "host", "level", "message"])
    writer.writerows(rows)
print("sample_data.csv generated")
```

Step 3: Read, filter and count the file

Write filter_errors.py to extract only ERROR-level rows using the csv module and pathlib, count every level, and append a UTC run-log entry (append, not overwrite) each time the script executes.

PYTHON

```
import csv, datetime
from pathlib import Path

src, dst = Path("sample_data.csv"), Path("errors_only.csv")
counts = {"INFO": 0, "WARN": 0, "ERROR": 0}
with src.open(newline="", encoding="utf-8") as inp, dst.open("w",
newline="", encoding="utf-8") as out:
    reader = csv.DictReader(inp)
    writer = csv.DictWriter(out, fieldnames=reader.fieldnames)
    writer.writeheader()
    for row in reader:
        counts[row["level"]] = counts.get(row["level"], 0) + 1
        if row["level"] == "ERROR":
            writer.writerow(row)
print(f"{counts['ERROR']} ERROR rows written to {dst};
{counts['WARN']} WARN rows seen; {counts['INFO']} INFO rows seen")

with open("run_history.log", "a", encoding="utf-8") as log:

log.write(f"{datetime.datetime.now(datetime.timezone.utc).isoformat(ti
mespec='seconds')} - filter_errors.py executed\n")
```

Step 4: Handle a missing file (exception primer)

Wrap a risky step in try/except and exit with a clear message instead of a traceback.

PYTHON

```
from pathlib import Path

src = Path("sample_data.csv")
try:
    text = src.read_text(encoding="utf-8")
except FileNotFoundError:
    raise SystemExit(f"Input file not found: {src}")
print(f"{len(text.splitlines())} lines read from {src}")
```

Step 5: Execute and verify

BASH

```
python generate_sample.py
python filter_errors.py
head errors_only.csv
cat run_history.log
```

Required Outcomes / Tasks

- sample_data.csv, errors_only.csv, and run_history.log submitted together, with a screenshot of the run (system clock visible).
- Extend filter_errors.py so it also writes warn_only.csv and prints the percentage of rows at each level; submit the modified script.
- Explain in 2–3 sentences why `with open(...)` is preferred over calling `open()/close()` manually.

Lab 3: Command-Line Tools with Python (argparse)

Course: DFSP-5208 | Scripting Programs | Week 3 | Program: BS Digital Forensics and Cyber Security

Title

Command-Line Tools with Python (argparse)

Objectives

- Build a reusable command-line utility using the free, built-in `argparse` module.
- Accept input file paths, flags, and options from the terminal instead of hard-coding them.
- Package the tool so classmates can run it with `--help` to see usage instructions.

Preamble / Short Theory

Hard-coded scripts (Lab 2) are useful once; command-line tools are reusable. `argparse`, included with every Python installation at no cost, turns a script into a small program with named options, default values, and automatic `--help` text, the same pattern used by professional forensic tooling. This lab converts the CSV filter built in Lab 2 into a flexible CLI utility.

Real tools also report what they do. Python's `logging` module (standard library) writes timestamped messages with a severity level, which is far more useful than print statements; Lab 12 returns to it in depth. This lab also begins the Git habit: every lab from now on ends with a commit.

Prerequisites

- Lab 2 completed.
- sample_data.csv from Lab 2 (or regenerate it).
- Git installed and configured (Lab 1, Step 7).

Step-by-Step Guide

Step 1: Create the CLI script

Create log_filter_cli.py accepting --input, --output, --level and --limit arguments. The corrected version counts the rows it writes in a plain loop; an earlier draft printed “0 rows written” because writer.writerow() returns the number of characters written, not None.

PYTHON

```
#!/usr/bin/env python3
"""Filter a CSV of log records by level. Usage: log_filter_cli.py --
input IN --output OUT [--level L] [--limit N]"""
import argparse, csv, logging, sys
from pathlib import Path

logging.basicConfig(level=logging.INFO,
format="%(asctime)s %(levelname)s %(message)s")


def main():
    parser = argparse.ArgumentParser(description="Filter CSV records
by log level.")
    parser.add_argument("--input", required=True, help="Path to source
CSV")
    parser.add_argument("--output", required=True, help="Path to write
filtered CSV")
    parser.add_argument("--level", default="ERROR", choices=["INFO",
"WARN", "ERROR"], help="Level to keep")
    parser.add_argument("--limit", type=int, default=0, help="Stop
after N matching rows (0 = no limit)")
    args = parser.parse_args()

    src, dst = Path(args.input), Path(args.output)
    if not src.exists():
        logging.error("Input file not found: %s", src)
        return 1
    dst.parent.mkdir(parents=True, exist_ok=True)

    written = skipped = 0
    with src.open(newline="", encoding="utf-8") as inp, dst.open("w",
newline="", encoding="utf-8") as out:
        reader = csv.DictReader(inp)
        writer = csv.DictWriter(out, fieldnames=reader.fieldnames)
        writer.writeheader()
        for line_no, row in enumerate(reader, start=2):       # line 1
is the header
            level = row.get("level")
            if level is None:
                skipped += 1
                logging.warning("line %d skipped: no 'level' column
value", line_no)
                continue
            if level == args.level:
                writer.writerow(row)
                written += 1
                if args.limit and written >= args.limit:
                    break
    print(f"{written} rows with level={args.level} written to {dst}
({skipped} skipped)")
```

```
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

Step 2: Run with --help

BASH

```
python log_filter_cli.py --help
```

Step 3: Run against the sample data

BASH

```
python log_filter_cli.py --input ../lab02/sample_data.csv --output
warn_only.csv --level WARN
python log_filter_cli.py --input ../lab02/sample_data.csv --output
out/warn3.csv --level WARN --limit 3
python log_filter_cli.py --input nope.csv --output x.csv; echo "exit
code: $?"   # message and exit code 1
```

The count printed on the last line must equal the number of data rows in the output file (wc -l minus the header line).

Step 4: Make the script executable (Linux/macOS)

The script already starts with the shebang line #!/usr/bin/env python3. Mark the file executable so it can be called directly, similar to a native command-line tool.

BASH

```
chmod +x log_filter_cli.py
./log_filter_cli.py --input ../lab02/sample_data.csv --output
warn_only.csv --level WARN
```

Step 5: Document the extra flag and commit

Document the --limit option in a short README.md and commit the lab to Git.

TEXT

```
# log_filter_cli
Filters a CSV of log records by level.

    python3 log_filter_cli.py --input IN.csv --output OUT.csv [--level
INFO|WARN|ERROR] [--limit N]

--limit N   stop after N matching rows (default 0 = no limit)
```

BASH

```
cd ~/forensic-scripting/lab03
git init
git add log_filter_cli.py README.md
git commit -m "Lab 3: argparse CLI with --limit"
```

Required Outcomes / Tasks

- log_filter_cli.py submitted with a screenshot of the --help output (system clock visible).
- Output CSV files for both ERROR and WARN levels, and the printed count matching wc `-l` minus one.
- README.md documenting --limit, plus one additional optional flag of your own choice (e.g. --host NAME).
- Screenshot of `git log --oneline` showing at least one commit for this lab.

Lab 4: Fundamentals of Bash Scripting

Course: DFSP-5208 | Scripting Programs | Week 4 | Program: BS Digital Forensics and Cyber Security

Title

Fundamentals of Bash Scripting

Objectives

- Write and run Bash scripts on Linux (native, WSL2, or macOS Terminal, all free).
- Use variables, conditionals, and loops to automate directory operations.
- Understand exit codes and basic error handling in shell scripts.

Preamble / Short Theory

Bash (Bourne Again SHell) ships free with every Linux distribution and macOS, and is available on Windows through WSL2 (Windows Subsystem for Linux, free and open source). Much of forensic tooling is glued together with shell scripts because Bash has direct, low-overhead access to the file-system and system utilities. This lab covers the syntax needed for the automation labs that follow.

Prerequisites

- A Bash shell: native on Linux/macOS, or WSL2 (`wsl --install`) on Windows 11.
- Basic familiarity with terminal navigation (cd, ls, mkdir).

Step-by-Step Guide

Step 1: Verify Bash version

BASH

```
bash --version   # GNU bash, version 5.x expected
```

Step 2: Create the lab folder

BASH

```
mkdir -p ~/forensic-scripting/lab04 && cd ~/forensic-scripting/lab04
```

Step 3: Write a directory-organiser script

Create organize.sh that creates dated sub-folders and moves files into them based on extension. The corrected script skips itself, counts items per category, handles names with spaces, and never overwrites an existing file (it prefixes the time instead, so both copies of evidence are kept).

BASH

```
#!/usr/bin/env bash
# organize.sh: move files in a folder into dated archive sub-folders
by extension.
# set -e: exit on any error; set -u: error on unset variables; set -o
pipefail: a pipeline fails if any stage fails.
set -euo pipefail

TARGET_DIR="${1:-.}"
[ -d "$TARGET_DIR" ] || { echo "Not a directory: $TARGET_DIR" >&2;
exit 1; }
DATE_TAG=$(date +%Y-%m-%d)
ARCHIVE="${TARGET_DIR}/archive_${DATE_TAG}"
SELF="$(basename "${BASH_SOURCE[0]}")"

mkdir -p "${ARCHIVE}/logs" "${ARCHIVE}/csv" "${ARCHIVE}/other"
n_logs=0; n_csv=0; n_other=0

move_safe() {            # never overwrite: on a name clash, prefix
the time so both copies are kept
  local src="$1" dest_dir="$2" base target
  base="$(basename "$src")"; target="${dest_dir}/${base}"
  [ -e "$target" ] && target="${dest_dir}/$(date +%H%M%S)_${base}"
  mv -- "$src" "$target"
}

for f in "${TARGET_DIR}"/*; do
  [ -f "$f" ] || continue
  [ "$(basename "$f")" = "$SELF" ] && continue          # never move
this script itself
  case "$f" in
    *.log) move_safe "$f" "${ARCHIVE}/logs";   n_logs=$((n_logs +
1)) ;;
    *.csv) move_safe "$f" "${ARCHIVE}/csv";     n_csv=$((n_csv +
1)) ;;
    *)     move_safe "$f" "${ARCHIVE}/other"; n_other=$((n_other +
1)) ;;
  esac
done

echo "Organised into ${ARCHIVE}: logs=${n_logs} csv=${n_csv}
other=${n_other}"
```

Step 4: Make it executable and test it

BASH

```
chmod +x organize.sh
touch test1.log test2.log "my file.log" test3.csv notes.txt
./organize.sh .
ls -R archive_*
```

Expected summary line: Organized into ./archive_YYYY-MM-DD: logs=3 csv=1 other=1. The script itself (organize.sh) must not have been moved.

Now test the name-clash protection: create another file called test1.log and run ./organize.sh . again. Both copies must survive, the new one with a time prefix.

Step 5: Explain the error handling

The comments in the script explain `set -euo pipefail`: exit on error (-e), exit on an unset variable (-u), and fail a pipeline if any stage fails (pipefail). Read them, then add one comment of your own describing what the move_safe function protects against.

Required Outcomes / Tasks

- organize.sh submitted along with a terminal screenshot of a successful run (system clock visible), showing the per-category counts.
- Extend the script with a fourth category (for example *.json into a json/ folder) and update the summary line; submit the modified script.
- Short written answer: what happens if `set -euo pipefail` is removed and the script hits an error midway? Give one example for each flag.

Lab 5: Automating System Tasks with Bash (cron)

Course: DFSP-5208 | Scripting Programs | Week 5 | Program: BS Digital Forensics and Cyber Security

Title

Automating System Tasks with Bash (cron)

Objectives

- Schedule a Bash script to run automatically using cron (free, built into Linux).
- Automate a timestamped backup of forensic working files.
- Read and interpret crontab syntax and system logs for scheduled jobs.

Preamble / Short Theory

Manually running backup or cleanup scripts is unreliable; cron a scheduler bundled with every Linux distribution at no cost, runs scripts unattended at fixed intervals. This is the same mechanism production servers use to run nightly backups, log rotation, and health checks. This lab builds a backup script and schedules it, mirroring how a forensic operations team would automate evidence/data preservation.

Every command also ends with an exit code: 0 means success and anything else means failure. Cron cannot show a terminal, so a scheduled script must report problems through its exit code and a log file. The backup script below checks its own archive and records a SHA-256 hash, so an altered or damaged backup is detected.

Prerequisites

- Lab 4 completed.
- Access to a Linux shell with cron installed and systemd running (`sudo apt install cron` on Ubuntu if missing). On WSL, enable systemd as described in How to Use This Manual.

Step-by-Step Guide

Step 1: Confirm cron is installed and running

BASH

```
sudo apt install -y cron
sudo systemctl enable --now cron
sudo systemctl status cron --no-pager
```

Step 2: Write the backup script

Create the lab folder and write backup.sh. The corrected script excludes .venv and node_modules (otherwise every run archives hundreds of megabytes), writes a SHA-256 checksum, verifies that the archive can be read, and deletes backups older than 7 days.

BASH

```
mkdir -p ~/forensic-scripting/lab05 && cd ~/forensic-scripting/lab05
```

BASH

```
#!/usr/bin/env bash
# backup.sh: timestamped, verified backup with 7-day retention. Paths
can be overridden for testing.
set -euo pipefail
SRC_DIR="${SRC_DIR:-$HOME/forensic-scripting}"
BACKUP_ROOT="${BACKUP_ROOT:-$HOME/forensic-backups}"
TS=$(date -u +%Y%m%d_%H%M%S)
ARCHIVE="${BACKUP_ROOT}/backup_${TS}.tar.gz"
mkdir -p "$BACKUP_ROOT"

tar --exclude='.venv' --exclude='node_modules' -czf "$ARCHIVE" -C
"$SRC_DIR" .
( cd "$BACKUP_ROOT" && sha256sum "$(basename "$ARCHIVE")" >>
SHA256SUMS )
tar -tzf "$ARCHIVE" > /dev/null                        # verify the
archive can be read
echo "$(date -u +%FT%TZ) backup_${TS}.tar.gz created and verified" >>
"$BACKUP_ROOT/backup_history.log"

find "$BACKUP_ROOT" -name 'backup_*.tar.gz' -mtime +7 -delete   #
retention: 7 days
```

Step 3: Make it executable and test manually

BASH

```
chmod +x ~/forensic-scripting/lab05/backup.sh
mkdir -p ~/forensic-scripting/.venv && echo test > ~/forensic-
scripting/.venv/junk   # a file that must NOT be backed up
~/forensic-scripting/lab05/backup.sh
echo "exit code: $?"
ls ~/forensic-backups
cd ~/forensic-backups && sha256sum -c SHA256SUMS
tar -tzf backup_*.tar.gz | grep -c venv   # expect 0
```

A second test shows a failure being reported: run SRC_DIR=/nonexistent ~/forensic-scripting/lab05/backup.sh; echo $? and confirm that the exit code is not 0.

Step 4: Schedule it with cron

Edit the crontab to run the backup every 10 minutes for demonstration purposes (a real deployment would use a nightly schedule). The output redirection matters: cron has no terminal, so any error message would otherwise be lost.

BASH

```
crontab -e
# add these lines, then save:
# demonstration: every 10 minutes
*/10 * * * * $HOME/forensic-scripting/lab05/backup.sh >>
$HOME/forensic-backups/cron.out 2>&1
# real use: nightly at 02:00
# 0 2 * * * $HOME/forensic-scripting/lab05/backup.sh >>
$HOME/forensic-backups/cron.out 2>&1
```

Step 5: Verify scheduled execution

BASH

```
crontab -l
# wait 20+ minutes, then:
cat ~/forensic-backups/backup_history.log
```

On WSL without systemd, run sudo service cron start after each restart; Windows students without WSL may use Task Scheduler (schtasks).

Required Outcomes / Tasks

- backup.sh submitted with the crontab entry shown in a screenshot (system clock visible).
- backup_history.log showing at least two automatic (unattended) runs about 10 minutes apart.
- Screenshot of `sha256sum -c SHA256SUMS` reporting OK, and of `grep -c venv` printing 0.
- Explain why the redirection `>> cron.out 2>&1` is needed in the crontab line, and what the retention line (`find ... -mtime +7 -delete`) does.

Lab 6: Web Scraping with Python (BeautifulSoup)

Course: DFSP-5208 | Scripting Programs | Week 6 | Program: BS Digital Forensics and Cyber Security

Title

Web Scraping with Python (BeautifulSoup)

Objectives

- Install and use BeautifulSoup4 and requests, both free, open-source Python libraries.
- Extract structured data from a public web page and store it as CSV.
- Respect robots.txt and apply polite scraping practices (rate limiting, User-Agent headers).

Preamble / Short Theory

Web scraping automates the collection of publicly available web data, in forensic work this ranges from gathering open-source threat intelligence to monitoring cloud-service status pages. `requests` fetches the raw HTML and `BeautifulSoup4` (both free/open source, installed via pip) parses it into a navigable tree so specific elements (tables, links, headlines) can be extracted programmatically instead of read by eye.

When a site offers an RSS or JSON feed (many status pages and threat-intelligence sources do), prefer the feed over scraping its HTML: it is more stable and puts less load on the server.

Prerequisites

- Labs 1–3 completed (Python + venv working).
- Internet access to reach a public test page (or use the local test page in Step 5 if the site is unreachable).

Step-by-Step Guide

Step 1: Install the libraries (free, from PyPI)

BASH

```
cd ~/forensic-scripting && mkdir lab06 && cd lab06
source ../lab01/.venv/bin/activate
pip install requests beautifulsoup4
```

Step 2: Check the target site's scraping policy

Before scraping any site, open its /robots.txt and confirm the path you intend to fetch is not disallowed. For this lab we use https://books.toscrape.com, a site built specifically for scraping practice.

Step 3: Write the scraper

The scraper identifies itself with a User-Agent, sets a timeout, handles network errors with try/except, tolerates a malformed item instead of crashing, and pauses between requests (a pause placed inside a loop over results that are already downloaded would not rate-limit anything).

PYTHON

```
import csv, sys, time
import requests
from bs4 import BeautifulSoup

URL = sys.argv[1] if len(sys.argv) > 1 else
"https://books.toscrape.com/"
HEADERS = {"User-Agent": "Lab06-Student-Scraper/1.0 (course
exercise)"}

try:
    resp = requests.get(URL, headers=HEADERS, timeout=10)
    resp.raise_for_status()
except requests.exceptions.Timeout:
    sys.exit(f"Timed out fetching {URL}")
except requests.exceptions.RequestException as exc:
    sys.exit(f"Request failed: {exc}")

soup = BeautifulSoup(resp.text, "html.parser")
rows = []
for book in soup.select("article.product_pod"):
    link = book.h3.a if book.h3 else None
    price = book.select_one(".price_color")
    if link is None or price is None:                    # tolerate a
missing element
        print("skipped one malformed item", file=sys.stderr)
        continue
    rows.append([link["title"], price.text.strip()])

with open("scraped_books.csv", "w", newline="", encoding="utf-8") as
f:
    writer = csv.writer(f)
    writer.writerow(["title", "price"])
    writer.writerows(rows)
print(f"Saved {len(rows)} records to scraped_books.csv")
time.sleep(1)                                             # one
request, then a polite pause before any next page
```

Step 4: Run and inspect the output

BASH

```
python scraper.py
head -5 scraped_books.csv
```

On the real site the first page returns 20 books.

Step 5: Test offline against a local page and a failure

Public sites are not always reachable. Create a tiny local page that contains one good item and one malformed item, serve it, and run the scraper against it. Then point the scraper at a port where nothing is listening to see the error handling.

BASH

```
mkdir -p testpage && cat > testpage/index.html <<'EOF'
<html><body>
<article class="product_pod"><h3><a title="Local Book One"
href="#">Local Book One</a></h3><p
class="price_color">&pound;10.00</p></article>
<article class="product_pod"><h3><a title="Broken Item"
href="#">Broken Item</a></h3></article>
</body></html>
EOF
python3 -m http.server 8765 --directory testpage &
python scraper.py http://127.0.0.1:8765/     # skipped one malformed
item, Saved 1 records
python scraper.py http://127.0.0.1:1/ ; echo "exit code: $?"   #
Request failed ..., exit code 1
kill %1
```

The same code works for a status or threat-intelligence page: change the URL and the CSS selectors, and prefer a feed when one exists.

Required Outcomes / Tasks

- scraper.py and scraped_books.csv submitted, with a screenshot of the run (system clock visible).
- Screenshot of the offline test (one record saved, one malformed item skipped) and of the failure case with its exit code.
- Extend the scraper to follow the “next” page link for up to 3 pages, with a 1-second pause between page requests.
- One paragraph explaining what robots.txt is and why rate limiting matters ethically and legally.

Lab 7: Node.js Scripting for File System Automation

Course: DFSP-5208 | Scripting Programs | Week 7 | Program: BS Digital Forensics and Cyber Security

Title

Node.js Scripting for File System Automation

Objectives

- Install a current Node.js LTS release (22 or 24; free, open source, OpenJS Foundation) and verify npm.
- Use Node's built-in `fs` and `path` modules to automate file renaming safely (dry run first, never overwrite).
- Compare JavaScript/Node.js scripting patterns with the Python equivalents from Lab 2.

Preamble / Short Theory

Node.js lets JavaScript run outside the browser, and is entirely free and open source. Many cloud-native and forensic tooling ecosystems (including serverless platforms used later in the course) run on Node.js, so this lab introduces its asynchronous file-handling model using the built-in `fs/promises` API, no paid packages required.

Prerequisites

- Administrative rights to install Node.js (current LTS: 22 or 24).
- Comfort with basic JavaScript syntax (variables, functions) is helpful but not required.
- Git installed and configured (Lab 1, Step 7).

Step-by-Step Guide

Step 1: Install the current Node.js LTS (22 or 24)

Do not pipe a downloaded script straight into a root shell. Use the snap package, or download the NodeSource set-up script, read it, and only then run it.

BASH

```
# Option A (simplest on Ubuntu/Kubuntu)
sudo snap install node --classic --channel=24

# Option B: download, inspect, then run
curl -fsSLo nodesource_setup.sh https://deb.nodesource.com/setup_24.x
less nodesource_setup.sh          # read it before running it
sudo -E bash nodesource_setup.sh
sudo apt install -y nodejs

node --version   # v22.x or v24.x
npm --version
```

Step 2: Create the project

BASH

```
mkdir -p ~/forensic-scripting/lab07 && cd ~/forensic-scripting/lab07
npm init -y
```

Step 3: Write a batch file-renamer

Create rename_forensic_files.js that adds a standardized timestamp prefix to every file in a folder. It is a dry run unless --apply is given, skips files that already carry a timestamp prefix, never overwrites, and exits with code 1 on an error.

JAVASCRIPT

```
// Usage: node rename_forensic_files.js <dir> [--apply]   (dry run by
default; never overwrites)
const fs = require('fs/promises');
const path = require('path');

const STAMPED = /^\d{4}-\d{2}-\d{2}T\d{2}-\d{2}-\d{2}-\d{3}Z_/;

async function renameFiles(dir, apply) {
  const stamp = new Date().toISOString().replace(/[:.]/g, '-');
  let count = 0, skipped = 0;
  for (const file of await fs.readdir(dir)) {
    const oldPath = path.join(dir, file);
    if (!(await fs.stat(oldPath)).isFile()) continue;
    if (STAMPED.test(file)) { skipped++; continue; }              //
already has a timestamp prefix
    const newPath = path.join(dir, `${stamp}_${file}`);
    try { await fs.access(newPath); console.error(`refusing to
overwrite ${newPath}`); continue; } catch {}
    console.log(`${apply ? 'RENAME' : 'DRY RUN'} ${file} ->
${path.basename(newPath)}`);
    if (apply) await fs.rename(oldPath, newPath);
    count++;
  }
  console.log(`${apply ? 'Renamed' : 'Would rename'} ${count} files,
skipped ${skipped}, in ${dir}`);
}

const args = process.argv.slice(2);
renameFiles(args.find(a => !a.startsWith('--')) || '.',
args.includes('--apply'))
  .catch(err => { console.error('Error:', err.message);
process.exit(1); });
```

Step 4: Test the script

BASH

```
mkdir sample_files && touch sample_files/a.txt sample_files/b.csv
node rename_forensic_files.js sample_files            # dry run: shows
what would change
node rename_forensic_files.js sample_files --apply    # renames the
files
node rename_forensic_files.js sample_files --apply    # second run:
renames 0, skips 2
ls sample_files
node rename_forensic_files.js nodir; echo "exit code: $?"   # error
message and exit code 1
```

Step 5: Practice the Git branch and pull-request workflow

Commit the working script on the main branch, then make your change on a feature branch. Push the branch to GitHub (free tier) or Gitea and open a pull request; ask a classmate to review and approve it.

BASH

```
git init
git add rename_forensic_files.js package.json
git commit -m "Lab 7: dry-run renamer"
git switch -c feature/prefix-option
# make your change (see Required Outcomes), then:
git add -A && git commit -m "Add --prefix option"
```

Required Outcomes / Tasks

- rename_forensic_files.js submitted with before/after directory listing screenshots (system clock visible), including the dry run.
- Extend the script with a `--prefix NAME` option that replaces the timestamp text in the new file name, on a feature branch, and submit the pull-request link or a screenshot of the reviewed pull request.
- Short comparison table (3 rows) contrasting Python's `pathlib`/`os` calls with Node's `fs`/`promises` equivalents.

Lab 8: Deploying Node.js Scripts as Serverless Functions

Course: DFSP-5208 | Scripting Programs | Week 8 | Program: BS Digital Forensics and Cyber Security

Title

Deploying Node.js Scripts as Serverless Functions

Objectives

- Understand the serverless/Functions-as-a-Service (FaaS) model without needing a paid cloud account.
- Write and test a Node.js forensic-alert function locally with a small harness that mimics the OpenFaaS Node template (Tier 1, required).
- Optionally deploy the same function with OpenFaaS on faasd, invoke it over HTTP and inspect its logs (Tier 2, optional).

Preamble / Short Theory

Serverless computing lets a function run on demand without managing a server. Rather than requiring a paid AWS/Azure subscription, this lab first writes and tests the function locally with a small harness (Tier 1, required and graded) and then, optionally, deploys it with OpenFaaS on faasd (Tier 2). faasd is a lightweight FaaS engine that needs only Linux with systemd and containerd, with no Kubernetes cluster. Check the License before installing it: the faasd Community Edition is licensed for non-commercial, personal use under the faasd CE EULA, so confirm with your department that classroom use is covered. The local harness has no such restriction. This mirrors how a forensic alerting function (for example, “flag any log entry containing a known IOC”) is built and tested before it is deployed.

Prerequisites

- Lab 7 completed (Node.js basics).
- Tier 1 needs only Node.js. Tier 2 (optional) also needs Docker (installed in Lab 1, Step 7), Linux with systemd, and faas-cli.

Step-by-Step Guide

Step 1: Create the lab folder and the local harness (Tier 1, required)

The harness loads a handler file and calls it the same way the OpenFaaS Node template does, so the function can be tested without faasd.

BASH

```
mkdir -p ~/forensic-scripting/lab08 && cd ~/forensic-scripting/lab08
```

JAVASCRIPT

```
// Local harness that mimics the OpenFaaS node template, so the
function can be tested without faasd.
const file = process.argv[2];
const handler = require(require('path').resolve(file));
const body = process.argv[3] ?? '';
const ctx = { status(code) { return { succeed: b => ({ status: code,
body: b }) }; } };
handler({ body }, ctx).then(r => console.log(JSON.stringify(r)));
```

Step 2: Write the function logic

Create handler.js. It is the same file you would later place in ioc-alert/handler.js for faasd.

JAVASCRIPT

```
'use strict'
const IOCS = ['malicious.exe', '203.0.113.99', 'suspicious-
domain.test'];   // add two or more of your own

module.exports = async (event, context) => {
  const text = (event.body || '').toString().toLowerCase();
  const hits = IOCS.filter(ioc => text.includes(ioc.toLowerCase()));
  return context.status(200).succeed({ scanned: true, hits, matched:
hits.length > 0 });
}
```

Step 3: Test the function

BASH

```
node harness.js handler.js 'beacon to 203.0.113.99 seen'
node harness.js handler.js 'all quiet'
```

Expected: the first command prints {"status":200,"body":{"scanned":true,"hits":["203.0.113.99"],"matched":true}} and the second prints a body with "hits":[] and "matched":false.

Step 4: Understand “stateless”

A serverless function keeps nothing between calls: every invocation starts fresh, so anything that must persist (alerts already sent, counters) has to live in a database or file outside the function. Lab 10 returns to this idea.

Step 5: Check the License and prerequisites (Tier 2, optional)

Only continue if your department confirmed that the faasd CE EULA covers classroom use, and your machine runs Linux with systemd (not plain Windows; use WSL2 with systemd enabled). Everything above this step is enough for full marks.

Step 6: Install faasd and faas-cli (Tier 2)

Follow the faasd handbook for your faasd version; the commands below are the ones used when this manual was revised. The faas-cli installer is downloaded and read before it is run.

BASH

```
git clone https://github.com/openfaas/faasd --depth=1
cd faasd
sudo ./hack/install.sh
sudo systemctl status faasd   # should show active (running)
cd ..

curl -sSLo get-faas-cli.sh https://cli.openfaas.com
less get-faas-cli.sh          # read it before running it
sudo sh get-faas-cli.sh
faas-cli version
```

Step 7: Start a local registry and scaffold the function (Tier 2)

faasd has no built-in registry, and faas-cli up builds and pushes the image, so run a local registry first. List the available Node templates and use the newest one (the node20 template is the oldest you should use: Node 20 reached end of life in April 2026).

BASH

```
docker run -d -p 5000:5000 --name registry registry:2
export OPENFAAS_URL=http://127.0.0.1:8080
sudo cat /var/lib/faasd/secrets/basic-auth-password | faas-cli login -
-username admin --password-stdin
faas-cli template store list
faas-cli template store pull <newest-node-template>
faas-cli new ioc-alert --lang <newest-node-template>
```

Step 8: Deploy, invoke and inspect logs (Tier 2)

Copy handler.js into ioc-alert/handler.js and open ioc-alert.yml. Change the image line to 127.0.0.1:5000/ioc-alert:latest, then build, deploy and invoke. Check the faasd handbook for the registry and pull-policy settings of your faasd version if the pull fails.

BASH

```
cp handler.js ioc-alert/handler.js
faas-cli up -f ioc-alert.yml
curl -X POST http://127.0.0.1:8080/function/ioc-alert -d 'connection
from 203.0.113.99 detected'   # expect "matched":true
faas-cli list
faas-cli logs ioc-alert
```

Required Outcomes / Tasks

- Harness output showing a successful IOC match (`matched: true`) and a clean case (`matched: false`), system clock visible.
- Add at least two additional IOC strings relevant to a sample scenario provided by the instructor, and resubmit handler.js with new test output.
- Short answer: what does “stateless” mean for a function, and why does it matter?
- Optional (Tier 2): screenshot of `faas-cli list` showing the deployed ioc-alert function and the curl output.

Lab 9: REST API Interaction: Fetching and Parsing JSON Data

Course: DFSP-5208 | Scripting Programs | Week 10 | Program: BS Digital Forensics and Cyber Security

Title

REST API Interaction: Fetching and Parsing JSON Data

Objectives

- Send GET requests to a REST API using Python `requests` and Node.js `fetch`/`axios` (free, OSS) and handle non-200 responses explicitly.
- Parse JSON responses and extract fields relevant to a forensic case timeline.
- Preserve each response as timestamped evidence with a SHA-256 hash and a custody-log entry, then store parsed records in a local SQLite database file.

Preamble / Short Theory

Forensic tooling frequently needs to pull data from external services, threat-intel feeds, case-management systems, breach-notification APIs. REST APIs exchange data as JSON over HTTP(S). Python's `requests` and Node's `axios` (installed free via pip/npm) are the de-facto open-source libraries for this. SQLite, built into Python's standard library and free everywhere, is used here in place of a paid cloud database so the lab requires no account or billing setup.

Evidence handling matters even for API data: save the raw response in a file whose name carries a UTC timestamp, record its SHA-256 hash, and append a line to a custody log. Anyone can then check later that the file has not changed. Never parse an error response as if it were data: check the status code first.

Prerequisites

- Labs 1–3 and 7 completed.
- Internet access to a public test API (https://jsonplaceholder.typicode.com), or the local mock API in Step 2 if it is unreachable.

Step-by-Step Guide

Step 1: Set up the project

BASH

```
cd ~/forensic-scripting && mkdir lab09 && cd lab09
source ../lab01/.venv/bin/activate
pip install requests
sudo apt install -y sqlite3     # command-line client used in Step 5
```

Step 2: Start the local mock API (optional fallback)

If the public API is unreachable, create mock_api.py and run it in a second terminal. Then point the scripts at it with the API_URL environment variable; leave API_URL unset to use the public site.

PYTHON

```
# mock_api.py - a tiny local stand-in for the public test API (no
internet needed)
import json
from http.server import BaseHTTPRequestHandler, HTTPServer

POSTS = [{"userId": (i % 5) + 1, "id": i, "title": f"case note {i}",
"body": f"details for note {i}"}
         for i in range(1, 101)]
TODOS = [{"userId": (i % 5) + 1, "id": i, "title": f"resource task
{i}", "completed": i % 3 == 0}
         for i in range(1, 101)]

class Handler(BaseHTTPRequestHandler):
    def do_GET(self):
        data = {"/posts": POSTS, "/todos": TODOS}.get(self.path)
        code = 200 if data is not None else 404
        body = json.dumps(data if data is not None else {"error": "not
found"}).encode()
        self.send_response(code)
        self.send_header("Content-Type", "application/json")
        self.send_header("Content-Length", str(len(body)))
        self.end_headers()
        self.wfile.write(body)

    def log_message(self, *args):      # keep the terminal quiet
        pass

if __name__ == "__main__":
    print("Mock API listening on http://127.0.0.1:8765 (Ctrl+C to
stop)")
    HTTPServer(("127.0.0.1", 8765), Handler).serve_forever()
```

BASH

```
python mock_api.py &
export API_URL=http://127.0.0.1:8765
```

Step 3: Write fetch_cases.py (evidence workflow)

The script checks the status code before parsing, saves the raw response as a timestamped evidence file, hashes it, writes a custody-log line, and stores 20 parsed records with a parameterized SQL statement.

PYTHON

```
import datetime, hashlib, json, os, sqlite3, sys
import requests

BASE = os.getenv("API_URL", "https://jsonplaceholder.typicode.com")
URL = f"{BASE}/posts"


def utc_stamp():
    return
datetime.datetime.now(datetime.timezone.utc).strftime("%Y%m%d_%H%M%S")


def fetch(url):
    resp = requests.get(url, timeout=10)
    if resp.status_code != 200:                                  # non-
200: report and stop, never parse an error body
        raise RuntimeError(f"HTTP {resp.status_code} from {url}:
{resp.text[:120]}")
    return resp.json()


def save_evidence(data, url):
    ts = utc_stamp()
    fname = f"evidence_{ts}.json"
    with open(fname, "w", encoding="utf-8") as f:
        json.dump(data, f, indent=2)
    with open(fname, "rb") as f:
        digest = hashlib.sha256(f.read()).hexdigest()
    with open("custody.log", "a", encoding="utf-8") as log:
        log.write(f"{ts}\t{url}\t{fname}\tSHA256={digest}\n")
    return fname, digest


def store(data):
    conn = sqlite3.connect("case_notes.db")
    with conn:
        conn.execute("""CREATE TABLE IF NOT EXISTS notes (
            id INTEGER PRIMARY KEY, user_id INTEGER, title TEXT, body
TEXT)""")
        for item in data[:20]:
            conn.execute("INSERT OR REPLACE INTO notes VALUES
(?,?,?,?)",
                         (item["id"], item["userId"], item["title"],
item["body"]))
    conn.close()
    return min(len(data), 20)


if __name__ == "__main__":
    try:
```

```
        data = fetch(URL)
        print(save_evidence(data, URL))
        print(f"{store(data)} records stored in case_notes.db")
    except requests.exceptions.RequestException as exc:
        sys.exit(f"Network failure: {exc}")
    except RuntimeError as exc:
        sys.exit(f"API failure: {exc}")
```

Step 4: Run it and verify the evidence

BASH

```
python fetch_cases.py
cat custody.log
sha256sum evidence_*.json     # must match the SHA256 value in
custody.log
```

Step 5: Query the stored data

BASH

```
sqlite3 case_notes.db "SELECT id, title FROM notes LIMIT 5;"
```

Step 6: Node.js equivalent with the built-in fetch

Node.js includes fetch, so no extra package is needed. Write fetch_cases.js:

JAVASCRIPT

```
const BASE = process.env.API_URL ||
'https://jsonplaceholder.typicode.com';

async function main() {
  try {
    const res = await fetch(`${BASE}/posts`, { signal:
AbortSignal.timeout(10000) });
    if (!res.ok) throw new Error(`HTTP ${res.status}`);          //
non-200 handled explicitly
    const data = await res.json();
    console.table(data.slice(0, 5).map(p => ({ id: p.id, title:
p.title })));
  } catch (err) {
    console.error('API error:', err.message);
    process.exitCode = 1;
  }
}
main();
```

If you prefer axios (free, OSS), install it with npm install axios and use this version, which also handles a non-200 status explicitly:

JAVASCRIPT

```
const url = (process.env.API_URL ||
'https://jsonplaceholder.typicode.com') + '/posts';
const axios = require('axios');
axios.get(url, { timeout: 10000, validateStatus: () => true })
  .then(res => {
    if (res.status !== 200) throw new Error(`HTTP ${res.status}`);
    console.table(res.data.slice(0, 5).map(p => ({ id: p.id, title:
p.title })));
  })
  .catch(err => { console.error('API error:', err.message);
process.exitCode = 1; });
```

Step 7: Test the failure paths

BASH

```
API_URL=http://127.0.0.1:8765/bad python fetch_cases.py; echo "exit
code: $?"   # HTTP 404 reported, exit code 1
API_URL=http://127.0.0.1:1 node fetch_cases.js; echo "exit code: $?"
# fetch failed, exit code 1
```

Step 8: Keys and rate limits

Real APIs need a key and limit how fast you may call them. Never write a key into the code: keep it in a .env file that .gitignore excludes, load it into the environment, and send it in a header. If the server answers 429 (too many requests), wait for the time in the Retry-After header and try again.

BASH

```
# .env  (never commit this file)
API_KEY=replace-with-your-key

set -a; source .env; set +a      # load it into the environment
```

PYTHON

```
import os, time
API_KEY = os.getenv("API_KEY")
headers = {"Authorization": f"Bearer {API_KEY}"} if API_KEY else {}
resp = requests.get(URL, headers=headers, timeout=10)
if resp.status_code == 429:
    time.sleep(int(resp.headers.get("Retry-After", "5")))
    resp = requests.get(URL, headers=headers, timeout=10)
```

Required Outcomes / Tasks

- One evidence_*.json file, custody.log showing its matching SHA-256, and case_notes.db submitted, with a screenshot of the sqlite3 query (system clock visible).
- fetch_cases.js console.table output screenshot.
- Screenshots showing both scripts handling a non-200 status or a network failure without crashing, and exiting with a non-zero code.

- Add API-key loading from .env and 429 handling (Step 8) to the Python script, and document the behavior in two sentences.

Lab 10: Automating Security Workflows with APIs

Course: DFSP-5208 | Scripting Programs | Week 11 | Program: BS Digital Forensics and Cyber Security

Title

Automating Security Workflows with APIs

Objectives

- Chain a file-scan step with an automated email alert using only free, open-source components.
- Use a local SMTP test server (Mailpit, or Python's aiosmtpd) as the mail sink, with no real mail credentials required.
- Make the workflow safe to schedule: remember what was already alerted in SQLite so the same finding is never e-mailed twice.

Preamble / Short Theory

Security workflows typically end with a notification. Rather than requiring a paid SMTP relay or real inbox credentials, a local SMTP sink captures every message: Mailpit shows them in a web UI, and Python's aiosmtpd prints them in the terminal. (MailHog, used in earlier editions, has had no release since 2020.) This lab connects the IOC-detection logic from Lab 8 to an automated alert email, the same pattern used in real Security Orchestration, Automation and Response (SOAR) pipelines. Two habits make such a workflow production-ready: send one consolidated message instead of one per finding, and keep state (here in SQLite) so that a scheduled job does not repeat itself.

Prerequisites

- Lab 8 completed (Tier 1 handler tested).
- Lab 9 completed (requests/fetch).
- Docker (installed in Lab 1) for Mailpit, or `pip install aiosmtpd` if Docker is not available.

Step-by-Step Guide

Step 1: Start a local SMTP sink (free, OSS)

Option A: Mailpit in Docker. Option B: Python's aiosmtpd, which prints each message in the terminal. Both listen on port 1025.

BASH

```
cd ~/forensic-scripting && mkdir lab10 && cd lab10
# Option A: Mailpit (web UI at http://localhost:8025)
docker run -d --name mailpit -p 1025:1025 -p 8025:8025 axllent/mailpit

# Option B: aiosmtpd (messages are printed in this terminal)
source ../lab01/.venv/bin/activate
pip install aiosmtpd
python -m aiosmtpd -n -l localhost:1025
```

Step 2: Write the alert workflow in Python

The script scans first, collects every match, and sends one consolidated email. If nothing matches it sends nothing; if the SMTP server is down it exits with a clear message.

PYTHON

```
import smtplib, sys
from email.mime.text import MIMEText

SMTP_HOST, SMTP_PORT = "localhost", 1025
IOCS = ["malicious.exe", "203.0.113.99"]
LOG_LINES = [
    "user admin logged in from 10.0.0.5",
    "outbound connection to 203.0.113.99 flagged",
    "process malicious.exe started by user bob",
    "scheduled backup completed",
    "dns query for example.org",
]


def scan_line(line):
    return [i for i in IOCS if i in line]


def send_summary(findings):
    body = "IOC summary\n" + "\n".join(f"- line {n}: {hits} -> {line}"
for n, hits, line in findings)
    msg = MIMEText(body)
    msg["Subject"] = f"Forensic Alert: {len(findings)} IOC match(es)"
    msg["From"], msg["To"] = "soc-automation@lab.local",
"analyst@lab.local"
    with smtplib.SMTP(SMTP_HOST, SMTP_PORT, timeout=10) as server:
        server.send_message(msg)


if __name__ == "__main__":
    findings = [(n, h, l) for n, l in enumerate(LOG_LINES, 1) if (h :=
scan_line(l))]
    if not findings:
        print("No IOC matches, no alert sent")
        sys.exit(0)
    try:
        send_summary(findings)                                  # ONE
consolidated email, not one per line
    except (OSError, smtplib.SMTPException) as exc:
        sys.exit(f"Alert failed: {exc}")
    print(f"Consolidated alert sent: {len(findings)} matches in
{len(LOG_LINES)} lines")
```

Step 3: Run the workflow and check the sink

```bash
python alert_workflow.py
# Mailpit: open http://localhost:8025 in a browser to view the
captured email
# aiosmtpd: the message appears in the terminal where it is running
```

Expected: one email listing every match, not one email per line.

Step 4: Node.js equivalent using nodemailer (free, OSS)

```bash
npm install nodemailer
```

Step 5: Write alert_workflow.js

```javascript
const nodemailer = require('nodemailer');
const IOCS = ['malicious.exe', '203.0.113.99'];
const LINES = ['user admin logged in', 'outbound connection to
203.0.113.99 flagged', 'process malicious.exe started', 'backup done',
'dns query'];

async function main() {
  const findings = LINES.map((l, i) => ({ n: i + 1, l, hits:
IOCS.filter(x => l.includes(x)) })).filter(f => f.hits.length);
  if (!findings.length) return console.log('No IOC matches, no alert
sent');
  const transporter = nodemailer.createTransport({ host: 'localhost',
port: 1025, secure: false });
  await transporter.sendMail({
    from: 'soc-automation@lab.local', to: 'analyst@lab.local',
    subject: `Forensic Alert: ${findings.length} IOC match(es)`,
    text: findings.map(f => `- line ${f.n}: ${f.hits.join(', ')} ->
${f.l}`).join('\n'),
  });
  console.log(`Consolidated alert sent: ${findings.length} matches`);
}
main().catch(e => { console.error('Alert failed:', e.message);
process.exit(1); });
```

Step 6: Add state and a schedule

A scheduled workflow must not repeat itself. Keep an alerts table in SQLite and skip any entity that was alerted in the last 24 hours. Add the check before sending and an INSERT after a successful send, then schedule the script with cron.

PYTHON

```
import sqlite3, datetime
conn = sqlite3.connect("alerts.db")
conn.execute("CREATE TABLE IF NOT EXISTS alerts (entity TEXT, ts TEXT,
action TEXT)")
def already_alerted(entity, hours=24):
    cutoff = (datetime.datetime.now(datetime.timezone.utc) -
datetime.timedelta(hours=hours)).isoformat()
    return conn.execute("SELECT 1 FROM alerts WHERE entity=? AND ts>?
LIMIT 1", (entity, cutoff)).fetchone() is not None
# after a successful send: conn.execute("INSERT INTO alerts VALUES
(?,?,?)", (entity, now_iso, "email")); conn.commit()
```

BASH

```
crontab -e
# add:
*/30 * * * * cd $HOME/forensic-scripting/lab10 && $HOME/forensic-
scripting/lab01/.venv/bin/python alert_workflow.py >> alert.out 2>&1
```

Run the script twice by hand first: the second run must report that the finding was already alerted and send no second email.

Required Outcomes / Tasks

- Screenshot of the mail sink (Mailpit web UI or aiosmtpd terminal) showing one consolidated alert email, system clock visible.
- Both alert_workflow.py and alert_workflow.js submitted.
- Extend either script to loop over 5 sample log lines and send one consolidated summary email instead of one per line (the provided script already does this; show it with your own five lines and your own IOC).
- Output of `SELECT * FROM alerts;` after the state extension, and a second run that sends no email.

Lab 11: Error Handling and Debugging in Scripts

Course: DFSP-5208 | Scripting Programs | Week 12 | Program: BS Digital Forensics and Cyber Security

Title

Error Handling and Debugging in Scripts

Objectives

- Apply try/except (Python) and try/catch (Node.js) to handle predictable failure conditions.
- Use the free, built-in VS Code debugger to step through a script and inspect variables.
- Produce scripts that fail gracefully with clear error messages instead of crashing.

Preamble / Short Theory

A forensic script that crashes mid-run on a single malformed log line can corrupt an entire evidence-processing job. Defensive coding, catching specific exceptions, validating input, and logging failures, is essential for reliability. VS Code's built-in debugger (free, no extension required for Python once the Python extension is installed) allows breakpoints, step-through execution, and live variable inspection without any paid tooling.

Prerequisites

- Lab 2 and Lab 9 completed.
- VS Code with the Python extension installed.
- Git installed and a remote on GitHub or Gitea (free) for the pull-request task.

Step-by-Step Guide

Step 1: Create the messy data file

Create messy_log.csv with three deliberately malformed rows (a word, an empty value and a value with a stray letter).

csv

```
timestamp,severity
2026-03-04T10:00:00,3
2026-03-04T10:01:00,high
2026-03-04T10:02:00,
2026-03-04T10:03:00,2
2026-03-04T10:04:00,9x
```

Step 2: Run a deliberately fragile script

PYTHON

```
import csv

def parse_row(row):
    return {
        "timestamp": row["timestamp"],
        "severity": int(row["severity"]),   # will crash on non-
numeric values
    }

with open("messy_log.csv") as f:
    for row in csv.DictReader(f):
        print(parse_row(row))
```

Run it and read the traceback: it stops at the first bad row with a ValueError.

Step 3: Add targeted exceptions, timestamped logging and a custom exception

Catch only the exceptions you expect (KeyError, ValueError, TypeError), never a bare except. Log with a timestamp so the log can be placed on a timeline. RetryExhausted is a custom exception raised when every attempt of a retried operation fails.

PYTHON

```
import csv, logging, time

logging.basicConfig(filename="parse_errors.log",
level=logging.WARNING,
                    format="%(asctime)s %(levelname)s %(message)s")


class RetryExhausted(Exception):
    """Raised when every attempt of a retried operation has failed."""


def parse_row(row):
    try:
        return {"timestamp": row["timestamp"], "severity":
int(row["severity"])}
    except (KeyError, ValueError, TypeError) as exc:
        logging.warning("Skipped malformed row %r: %s", row, exc)
        return None


def with_retry(fn, attempts=3, delay=1.0, retry_on=(OSError,)):
    """Retry fn on transient errors. requests' exceptions subclass
OSError, NOT the builtin ConnectionError."""
    for i in range(1, attempts + 1):
        try:
            return fn()
        except retry_on as exc:
            logging.warning("Attempt %d/%d failed: %s", i, attempts,
exc)
            time.sleep(delay * 2 ** (i - 1))
    raise RetryExhausted(f"all {attempts} attempts failed")


if __name__ == "__main__":
    results = []
    try:
        with open("messy_log.csv", newline="", encoding="utf-8") as f:
            for row in csv.DictReader(f):
                parsed = parse_row(row)
                if parsed:
                    results.append(parsed)
    except FileNotFoundError:
        logging.error("messy_log.csv not found")
        raise SystemExit("messy_log.csv not found")
    print(f"Successfully parsed {len(results)} rows; see
parse_errors.log for skipped rows")
```

Expected: Successfully parsed 2 rows; see parse_errors.log for skipped rows. The log lines start with a date and time, for example 2026-10-09 10:53:38,467 WARNING Skipped malformed row ...

Step 4: Demonstrate the custom exception

Add this to the end of the script (inside the if __name__ block or in a separate file) and run it: every attempt fails, the retries are logged, and RetryExhausted is raised and caught.

PYTHON

```
def always_fails():
    raise OSError("simulated outage")

try:
    with_retry(always_fails, attempts=3, delay=0.1)
except RetryExhausted as exc:
    print("giving up:", exc)
```

Step 5: Debug with VS Code and with pdb

In VS Code, click to the left of the `return` line inside parse_row to set a breakpoint, press F5 (Run ▸ Start Debugging), open the Variables panel and step with F10. When the value of the bad field is visible, take your screenshot.

The same job in the terminal needs no editor: b parse_row sets a breakpoint, c continues, p row prints the row, n steps to the next line.

BASH

```
python3 -m pdb parse_messy.py
(Pdb) b parse_row
(Pdb) c
(Pdb) p row
(Pdb) n
(Pdb) c
```

Step 6: Node.js equivalent

In Node.js the same defensive pattern uses a check (or try/catch) and a timestamped warning:

JAVASCRIPT

```
function parseRow(row) {
  const sev = Number(row.severity);
  if (row.severity === '' || Number.isNaN(sev)) {
    console.warn(`${new Date().toISOString()} WARN skipped row:
invalid severity "${row.severity}"`);
    return null;
  }
  return { timestamp: row.timestamp, severity: sev };
}
```

Add a few lines to run it over messy_log.csv:

JAVASCRIPT

```
const fs = require('fs');
const [, ...lines] = fs.readFileSync('messy_log.csv',
'utf8').trim().split('\n');
const rows = lines.map(l => { const [timestamp, severity] =
l.split(','); return { timestamp, severity }; });
const parsed = rows.map(parseRow).filter(Boolean);
console.log(`Successfully parsed ${parsed.length} rows`);
```

Step 7: Review your change through a pull request

Commit the corrected script on a branch, push it, open a pull request and ask a classmate to review it. Reviewers check that no bare except remains and that every skipped row is logged.

BASH

```
cd ~/forensic-scripting/lab11
git init
git add parse_messy.py messy_log.csv
git commit -m "Lab 11: tolerant parser"
git switch -c feature/tolerant-parser
# push the branch and open the pull request on GitHub or Gitea
```

Required Outcomes / Tasks

- messy_log.csv (with at least 3 deliberately malformed rows) and the corrected script submitted.
- parse_errors.log showing timestamped lines for the malformed rows being caught, not crashing the program.
- Screenshot of the VS Code debugger paused at a breakpoint with the Variables panel visible, or a pdb transcript showing `p row`, with the system clock visible.
- Screenshot of the RetryExhausted demonstration, and the link to your reviewed pull request.
- Two sentences: why is `except (KeyError, ValueError)` better than a bare `except`?

Lab 12: Logging and Monitoring in Security Scripts

Course: DFSP-5208 | Scripting Programs | Week 13 | Program: BS Digital Forensics and Cyber Security

Title

Logging and Monitoring in Security Scripts

Objectives

- Replace ad-hoc print statements with Python's `logging` module and Node's `winston` (both free/OSS).
- Configure log levels, rotation, console output and structured (JSON) output.
- Make a log tamper-evident with a SHA-256 hash, and view and filter logs quickly using `lnav` (free, open-source log file navigator).

Preamble / Short Theory

Print statements do not scale: they cannot be filtered by severity, timestamped consistently, or redirected without editing code. Python's `logging` module (standard library, free) and Node's `winston` package (MIT-licensed, free) provide leveled, structured logging with rotation support, matching what is expected of production-grade forensic tooling. `lnav` is a free terminal-based log viewer that auto-detects formats and supports live filtering.

Prerequisites

- Lab 11 completed.
- lnav installed (`sudo apt install lnav`).

Step-by-Step Guide

Step 1: Configure Python logging with rotation, JSON output and a console handler

One logger writes a text log, a JSON-lines log (one JSON object per line, easy for tools to read) and prints warnings and errors to the console. Both files rotate at 1 MB, keeping 3 old files.

BASH

```
cd ~/forensic-scripting && mkdir lab12 && cd lab12 &&
source ../lab01/.venv/bin/activate
```

PYTHON

```
import datetime, json, logging, sys
from logging.handlers import RotatingFileHandler


class JsonFormatter(logging.Formatter):
    def format(self, r):
        return json.dumps({"ts":
datetime.datetime.fromtimestamp(r.created,
datetime.timezone.utc).isoformat(),
                           "level": r.levelname, "msg":
r.getMessage(), "logger": r.name,
                           "context": getattr(r, "context", {})})


def make_logger(name="pipeline", max_bytes=1_000_000):
    logger = logging.getLogger(name)
    logger.setLevel(logging.DEBUG)
    text = RotatingFileHandler("pipeline.log", maxBytes=max_bytes,
backupCount=3, encoding="utf-8")
    text.setFormatter(logging.Formatter("%(asctime)s | %(levelname)s
| %(message)s"))
    jsonl = RotatingFileHandler("pipeline.jsonl", maxBytes=max_bytes,
backupCount=3, encoding="utf-8")
    jsonl.setFormatter(JsonFormatter())
    console = logging.StreamHandler(sys.stderr)
    console.setLevel(logging.WARNING)
    for h in (text, jsonl, console):
        logger.addHandler(h)
    return logger


if __name__ == "__main__":
    log = make_logger()
    log.info("Pipeline started", extra={"context": {"case": "C-001"}})
    log.warning("Suspicious file skipped: corrupt_evidence.dat")
    log.error("Failed to hash file: permission denied")
```

Step 2: Run it and read the three outputs

BASH

```
python logger_demo.py
cat pipeline.log
cat pipeline.jsonl
```

Expected: the text log has three lines with a time stamp and level, the JSON log has the same events as JSON objects, and only the WARNING and ERROR messages appear on the console.

Step 3: Install and configure winston (Node.js)

BASH

```
npm install winston
```

Step 4: Write logger.js with built-in rotation

Winston's File transport already rotates with maxsize and maxFiles, so no extra package is needed (winston-daily-rotate-file is also acceptable if you prefer daily files).

JAVASCRIPT

```
const winston = require('winston');
const logger = winston.createLogger({
  level: 'debug',
  format: winston.format.combine(winston.format.timestamp(),
winston.format.json()),
  transports: [
    new winston.transports.File({ filename: 'pipeline.json.log',
maxsize: 1_000_000, maxFiles: 3 }),
    new winston.transports.Console(),
  ],
});
logger.info('Pipeline started');
logger.warn('Suspicious file skipped: corrupt_evidence.dat');
logger.error('Failed to hash file: permission denied');
```

Step 5: Make a closed log tamper-evident

Lower max_bytes in the Python script to about 2000 and write 100 extra log lines in a loop so that the log rotates and a closed file such as pipeline.log.1 appears. Hash that closed file, store the hash somewhere else, and verify it later. The live pipeline.log changes with every line, so it must never be hashed while it is open.

BASH

```
# set maxBytes small (e.g. 2000) so the log rotates, then hash a
CLOSED file:
sha256sum pipeline.log.1 >> log_hashes.txt
# later, verify:
sha256sum -c log_hashes.txt        # OK, or FAILED if one character
was changed
```

Change one character in pipeline.log.1 and run the verification again: it must report FAILED.

Step 6: Inspect logs with lnav

BASH

```
lnav pipeline.log
# inside lnav: press e / E to jump to the next / previous error line,
# or type  :filter-in WARNING|ERROR  to show only those lines
```

Key bindings vary slightly between lnav versions; take the screenshot of the filtered view.

Required Outcomes / Tasks

- pipeline.log, pipeline.jsonl and pipeline.json.log submitted, with at least one message at each of the INFO, WARNING and ERROR levels.
- Screenshot of lnav showing a filtered view (only WARNING/ERROR lines), with the system clock visible.
- A rotated file (pipeline.log.1), its hash recorded in log_hashes.txt, and a screenshot of `sha256sum -c log_hashes.txt` showing OK, then FAILED after you change one character.
- Configure winston so that no single file exceeds 1 MB (maxsize and maxFiles, or winston-daily-rotate-file) and show the setting.

Lab 13: Introduction to Docker: Containerizing Forensic Tools

Course: DFSP-5208 | Scripting Programs | Week 14 | Program: BS Digital Forensics and Cyber Security

Title

Introduction to Docker: Containerizing Forensic Tools

Objectives

- Install Docker Engine Community Edition (Apache-2.0, free and open source).
- Write a Dockerfile that packages a forensic Python script and its dependencies.
- Build and run the container, confirming it produces identical output to the host run.

Preamble / Short Theory

Containerization solves the “works on my machine” problem by packaging code with its exact runtime and dependencies. Docker Engine CE is free and open source (Apache-2.0 license); this differs from Docker Desktop, which is free only for personal, small-business and educational use, so this course standardizes on Docker Engine CE on Linux/WSL2 so every student uses a fully open, unrestricted toolchain. Podman is a fully open-source, daemon-less drop-in alternative for students who prefer it.

One rule prevents the most common failure in this lab: a bind mount (-v) replaces whatever the image has at that path with the host folder. Keep the code in /app and mount only a separate output folder (/app/out). The container also runs as a non-root user, which limits the damage if the program is ever compromised.

Prerequisites

- Lab 1, Step 7 completed (Docker installed and `docker run --rm hello-world` works).
- A working corrected script from Lab 3 (and sample_data.csv from Lab 2).

Step-by-Step Guide

Step 1: Confirm Docker Engine CE

BASH

```
docker --version
docker run --rm hello-world
# Fully open-source daemon-less alternative: sudo apt install podman
```

Step 2: Prepare the application

BASH

```
mkdir -p ~/forensic-scripting/lab13 && cd ~/forensic-scripting/lab13
cp ../lab03/log_filter_cli.py .
cp ../lab02/sample_data.csv .
```

Step 3: Write the Dockerfile and .dockerignore

The image is small, creates a non-root user, and keeps the output folder separate from the code. No requirements.txt is needed because the script uses only the standard library. Use python:3.13-slim if that is what your lab PC provides; the version is not important for the exercise.

DOCKERFILE

```
FROM python:3.12-slim
WORKDIR /app
COPY log_filter_cli.py sample_data.csv ./
RUN useradd --create-home appuser && mkdir -p /app/out && chown
appuser /app/out
USER appuser
ENTRYPOINT ["python", "log_filter_cli.py"]
CMD ["--input", "sample_data.csv", "--output", "/app/out/output.csv",
"--level", "ERROR"]
```

Add a .dockerignore file so that the virtual environment, Git data, secrets, logs and earlier output never enter the image:

TEXT

```
.venv/
.git/
__pycache__/
.env
*.log
out/
```

Step 4: Build the image

BASH

```
docker build -t forensic-log-filter:1.0 .
```

Step 5: Run the container and retrieve the output

Mount only the out sub-folder. Mounting a host folder over /app would hide log_filter_cli.py and the run would fail with “can't open file”.

BASH

```
docker build -t forensic-log-filter:1.0 .
mkdir -p out
docker run --rm --user "$(id -u):$(id -g)" -v "$PWD/out:/app/out"
forensic-log-filter:1.0
cat out/output.csv | head
# PowerShell: -v "${PWD}/out:/app/out" (and omit --user)
docker images forensic-log-filter
docker history forensic-log-filter:1.0
```

Expected: the console prints N rows with level=ERROR written to /app/out/output.csv (0 skipped), and out/output.csv appears on the host. Compare it with a host run; the two hashes must match:

BASH

```
python log_filter_cli.py --input sample_data.csv --output
host_output.csv --level ERROR
sha256sum out/output.csv host_output.csv
```

Step 6: Inspect the image

BASH

```
docker images forensic-log-filter
docker history forensic-log-filter:1.0
```

Step 7: Prove the build cache, .dockerignore and --env-file

Build twice without changing anything: every layer is reused (shown as CACHED). Then change only log_filter_cli.py and rebuild: layers before the COPY line stay cached, which is why files that change often are copied late. Finally pass a setting with --env-file.

BASH

```
docker build -t forensic-log-filter:1.0 .        # run it a second
time: layers show CACHED
printf 'LEVEL=WARN\n' > .env
docker run --rm --env-file .env --entrypoint sh forensic-log-
filter:1.0 -c 'echo "LEVEL is $LEVEL"'
```

To prove that .dockerignore works, create a file named secret.env, rebuild, and confirm that `docker run --rm --entrypoint ls forensic-log-filter:1.0 /app` does not list it (add secret.env to .dockerignore first).

Required Outcomes / Tasks

- Dockerfile and .dockerignore submitted along with `docker images` output showing the built image (system clock visible).
- out/output.csv produced by the containerized run, with `sha256sum` showing that it matches the host-run output from Lab 3.
- Screenshot of the second build showing CACHED layers, and of the --env-file demonstration.
- One paragraph: why is `python:3.12-slim` preferred over the full `python:3.12` image for this container?

Lab 14: Automating Container Deployment

Course: DFSP-5208 | Scripting Programs | Weeks 14–15 | Program: BS Digital Forensics and Cyber Security

Title

Automating Container Deployment

Objectives

- Run a free, self-hosted Docker Registry (open source, Apache-2.0) instead of a paid Docker Hub plan.
- Tag and push the Lab 13 image to the local registry.
- Pull and run the image from the registry on a second (simulated) host.

Preamble / Short Theory

Docker Hub's free tier has pull-rate limits and a limited number of private repositories (check the current terms). The official `registry:2` image is fully open source and can be self-hosted at zero cost, which is what many organisations do internally for sensitive forensic tool images that must never leave a controlled network. This lab stands up a local registry and treats it as the deployment target. The lab registry has no TLS and no authentication, so use it only on your own machine or a closed lab network.

Prerequisites

- Lab 13 completed (forensic-log-filter:1.0 image built).

Step-by-Step Guide

Step 1: Run a local open-source registry

BASH

```
docker run -d -p 5000:5000 --restart=always --name local-registry
registry:2
```

Step 2: Tag the image for the local registry

BASH

```
docker tag forensic-log-filter:1.0 localhost:5000/forensic-log-
filter:1.0
```

Step 3: Push the image

BASH

```
docker push localhost:5000/forensic-log-filter:1.0
```

Step 4: Verify it is stored in the registry

BASH

```
curl http://localhost:5000/v2/_catalog
#   {"repositories":["forensic-log-filter"]}
curl http://localhost:5000/v2/forensic-log-filter/tags/list
#   {"name":"forensic-log-filter","tags":["1.0"]}
```

Step 5: Simulate a second host: remove the local image, then pull it back and run it

The run mounts only a new out2 sub-folder, never /app.

BASH

```
docker rmi forensic-log-filter:1.0 localhost:5000/forensic-log-
filter:1.0
docker pull localhost:5000/forensic-log-filter:1.0
mkdir -p out2
docker run --rm --user "$(id -u):$(id -g)" -v "$PWD/out2:/app/out"
localhost:5000/forensic-log-filter:1.0
```

Required Outcomes / Tasks

- Output of `curl http://localhost:5000/v2/_catalog` showing the pushed image (system clock visible).
- Screenshot of the pull-then-run sequence proving the image was retrieved fresh from the registry, and the file out2/output.csv.
- Short note on one advantage and one limitation of self-hosting a registry versus using Docker Hub's free tier.

Lab 15: CI/CD Pipeline Automation for Secure Deployment

Course: DFSP-5208 | Scripting Programs | Week 15 | Program: BS Digital Forensics and Cyber Security

Title

CI/CD Pipeline Automation for Secure Deployment

Objectives

- Build a CI/CD pipeline that builds, tests, scans for vulnerabilities and pushes the tool image automatically on every commit.
- Route A: run Jenkins (free, open source, self-hosted) with a Jenkinsfile that polls the repository.
- Route B: write the equivalent GitHub Actions workflow, which is the route used in the lecture notes and in Assignment 3.

Preamble / Short Theory

Continuous Integration/Continuous Deployment (CI/CD) automates the build-test-deploy cycle so every code change is validated the same way, every time. A security-minded pipeline adds a vulnerability scan (Trivy) that fails the build when serious problems are found. Jenkins is free, open source (MIT license) and self-hostable, so it needs no account limits or paid seats; GitHub Actions has a free tier and is what the course notes and Assignment 3 use. Complete Route A (Jenkins) and then Route B, which takes about 15 minutes. Both wire the pipeline to the Git repository and local registry built in Labs 13–14, mirroring real DevSecOps practice.

Prerequisites

- Labs 13–14 completed (registry running on localhost:5000).
- Git installed; the lab13 folder will become the repository.
- Docker running (Jenkins runs as a container). Route B needs a free GitHub account.

Step-by-Step Guide

Step 1: Route A: initialise the Git repository

The Jenkinsfile belongs in the lab13 repository, next to the Dockerfile.

BASH

```
cd ~/forensic-scripting/lab13
git init
git add Dockerfile .dockerignore log_filter_cli.py sample_data.csv
git commit -m 'Initial tool'
```

Step 2: Write the Jenkinsfile (build, test, scan, push)

The pipeline polls the repository every two minutes, builds the image, runs it as a test, scans it with Trivy and only then pushes. Test comes first so a broken image is never pushed; `--exit-code 1` makes the scan stage fail the build when HIGH or CRITICAL vulnerabilities with a fix are found. Replace <PINNED_VERSION> with the Trivy release your instructor announces (pinning a version avoids surprise changes).

GROOVY

```
pipeline {
  agent any
  triggers { pollSCM('H/2 * * * *') }
  environment {
    IMAGE = "localhost:5000/forensic-log-filter"
    TAG = "${env.BUILD_NUMBER}"
  }
  stages {
    stage('Build') { steps { sh 'docker build -t $IMAGE:$TAG .' } }
    stage('Test') {
      steps { sh 'docker run --rm $IMAGE:$TAG --input sample_data.csv
--output /tmp/out.csv --level ERROR' }
    }
    stage('Scan') {
      steps {
        sh 'docker run --rm -v
/var/run/docker.sock:/var/run/docker.sock
aquasec/trivy:<PINNED_VERSION> image --exit-code 1 --severity
HIGH,CRITICAL --ignore-unfixed $IMAGE:$TAG'
      }
    }
    stage('Push') { steps { sh 'docker push $IMAGE:$TAG' } }
  }
  post {
    success { echo 'Pipeline completed: image built, tested, scanned
and pushed.' }
    failure { echo 'Pipeline failed: check the console output.' }
  }
}
```

BASH

```
cd ~/forensic-scripting/lab13
git add Jenkinsfile && git commit -m 'Add Jenkinsfile'
```

Step 3: Build a Jenkins image that contains the Docker CLI

The official Jenkins image has no Docker CLI, so docker build would fail with “docker: not found”. Build a small image on the lts-jdk21 tag (Jenkins now requires Java 21).

DOCKERFILE

```
# Dockerfile.jenkins
FROM jenkins/jenkins:lts-jdk21
USER root
COPY --from=docker:cli /usr/local/bin/docker /usr/local/bin/docker
USER jenkins
```

BASH

```
docker build -f Dockerfile.jenkins -t jenkins-docker:lts .
```

Step 4: Run Jenkins

The container needs three things: the Docker socket (with its group), the repository as a read-only folder it can see, and a volume for its own data.

BASH

```
docker build -f Dockerfile.jenkins -t jenkins-docker:lts .
docker volume create jenkins_home
docker run -d --name jenkins -p 8080:8080 -p 50000:50000 \
  -v jenkins_home:/var/jenkins_home \
  -v /var/run/docker.sock:/var/run/docker.sock \
  --group-add "$(stat -c %g /var/run/docker.sock)" \
  -v "$HOME/forensic-scripting/lab13:/repo:ro" \
  jenkins-docker:lts
docker exec jenkins git config --global --add safe.directory /repo
docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword
```

Step 5: Complete setup in the browser

Open http://localhost:8080, paste the initial admin password, choose “Install suggested plugins” (all free/OSS), and create an admin user.

Step 6: Create the pipeline job and run it

New Item ▸ Pipeline ▸ name it forensic-log-filter-pipeline ▸ under Pipeline choose “Pipeline script from SCM” ▸ SCM: Git ▸ Repository URL /repo (or a GitHub/Gitea URL) ▸ Script Path Jenkinsfile. Choose Build Now, then make a small change, commit it, and wait for the poll to start a build by itself.

BASH

```
# trigger from the Jenkins UI: 'Build Now', then prove the commit
trigger:
cd ~/forensic-scripting/lab13
echo '# change' >> log_filter_cli.py && git commit -am 'Trigger build'
curl http://localhost:5000/v2/forensic-log-filter/tags/list     # the
new build number appears
```

Step 7: Route B: the same pipeline in GitHub Actions

Create .github/workflows/ci.yml in your repository, push to GitHub, and open the Actions tab. Permissions are limited to reading the repository, and the Trivy action should be pinned to a released tag or commit that you have checked; replace <PINNED_TAG_OR_SHA> and check the current major versions of actions/checkout and actions/setup-python.

YAML

```
name: CI
on:
  push:
  pull_request:
permissions:
  contents: read
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - run: python -m py_compile log_filter_cli.py
      - run: python log_filter_cli.py --input sample_data.csv --output
/tmp/out.csv --level ERROR
  build-and-scan:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: docker build -t forensic-log-filter:${{ github.sha }} .
      - uses: aquasecurity/trivy-action@<PINNED_TAG_OR_SHA>
        with:
          image-ref: forensic-log-filter:${{ github.sha }}
          severity: CRITICAL,HIGH
          ignore-unfixed: true
          exit-code: "1"
```

Step 8: Protect the main branch

In the repository settings add a branch protection rule for main: require a pull request and require the CI checks to pass before merging. Then open a pull request that breaks the build on purpose and show that it cannot be merged.

Required Outcomes / Tasks

- Jenkinsfile submitted alongside a screenshot of a green pipeline run showing the Build, Test, Scan and Push stages, with the system clock visible.
- Console output log from the Test stage showing the containerised script executed correctly.
- Confirm via `curl http://localhost:5000/v2/forensic-log-filter/tags/list` that the new build tag was pushed.
- Route B: ci.yml submitted with a screenshot of a green GitHub Actions run, and a screenshot of the branch-protection rule.
- Short answer: why does Test come before Build/Push, and what does `exit-code: 1` do in the scan stage?

Lab 16: Capstone Lab: Designing and Deploying a Complete Forensic Pipeline

Course: DFSP-5208 | Scripting Programs | Week 16 | Program: BS Digital Forensics and Cyber Security

Title

Capstone Lab: Designing and Deploying a Complete Forensic Pipeline

Objectives

- Integrate scripting, error handling, logging, containerization, and CI/CD from Labs 1–15 into one working pipeline.
- Design a pipeline that ingests raw log data, filters it, flags IOC matches, alerts via email, and archives results.
- Present and defend design decisions in a short demo to the class/instructor.

Preamble / Short Theory

The capstone consolidates the semester's tools into one forensic evidence-processing pipeline end to end: a Bash orchestration script (Lab 5) runs a Python filtering script (Labs 2–3) with timestamped logging (Lab 12) and defensive error handling (Lab 11); the filtered data is scanned by the IOC function (Lab 8, through the local harness or faasd); the scan result is read by an alert script that e-mails through the SMTP sink (Lab 10); the results are archived with a SHA-256 hash; and the whole toolchain is containerized (Lab 13), pushed to the local registry (Lab 14), and built automatically by CI/CD (Lab 15). Every stage reads exactly what the previous stage wrote, and all components remain free and open source.

Prerequisites

- Labs 1–15 completed and their artifacts available.
- The SMTP sink from Lab 10 running (Mailpit or aiosmtpd on port 1025).
- A short (5–10 minute) in-class or recorded demo slot booked with the instructor.

Step-by-Step Guide

Step 1: Design the pipeline

Sketch a diagram (hand-drawn, draw.io free tier, or the OSS tool Excalidraw) showing data flow: raw logs → filter script → IOC scan → alert → archive → CI/CD. Submit the diagram alongside the code.

Step 2: Assemble the project repository

Copy the files from earlier labs. The Jenkinsfile and Dockerfile come from lab13 (that is where Lab 15 created them).

BASH

```
mkdir -p ~/forensic-scripting/capstone && cd ~/forensic-
scripting/capstone
git init
mkdir -p data scripts faas docker jenkins .github/workflows out
cp ../lab03/log_filter_cli.py scripts/
cp ../lab08/handler.js ../lab08/harness.js faas/
cp ../lab13/Dockerfile docker/Dockerfile.lab13
cp ../lab13/Jenkinsfile jenkins/
cp ../lab13/.github/workflows/ci.yml .github/workflows/ 2>/dev/null ||
true   # only if you completed Route B
```

Target layout:

TEXT

```
capstone/
  data/raw.csv
  scripts/log_filter_cli.py     (Lab 3)
  scripts/alert_from_scan.py
  faas/handler.js  faas/harness.js   (Lab 8)
  docker/Dockerfile   jenkins/Jenkinsfile   .github/workflows/ci.yml
  run_pipeline.sh   diagram.png   README.md
```

Step 3: Supply the input data

The pipeline needs data/raw.csv. Create it with this content:

CSV

```
timestamp,host,level,message
2026-03-04T10:00:00,host-1,INFO,user admin logged in
2026-03-04T10:01:00,host-2,ERROR,outbound connection to 203.0.113.99
flagged
2026-03-04T10:02:00,host-3,ERROR,disk almost full
2026-03-04T10:03:00,host-2,WARN,process malicious.exe started
```

Step 4: Write the alert script (it reads the scan result)

In the earlier draft the alert script ignored the scan result and used a hard-coded line, so the stages were not connected. This version reads the JSON written by the scan stage and e-mails only when it reports a match.

PYTHON

```
import json, os, smtplib, sys
from email.mime.text import MIMEText

path = sys.argv[1]
with open(path, encoding="utf-8") as f:
    result = json.load(f)
result = result.get("body", result)            # accept the raw
function body or the harness envelope
if not result.get("matched"):
    print("No IOC matches, no alert sent")
    sys.exit(0)

msg = MIMEText(f"IOC hits: {result['hits']}")
msg["Subject"] = "Forensic Alert: IOC Match"
msg["From"], msg["To"] = "soc-automation@lab.local",
"analyst@lab.local"
host, port = os.getenv("MAIL_HOST", "localhost"),
int(os.getenv("MAIL_PORT", "1025"))
try:
    with smtplib.SMTP(host, port, timeout=10) as server:
        server.send_message(msg)
except (OSError, smtplib.SMTPException) as exc:
    sys.exit(f"Alert failed: {exc}")
print("Alert sent:", result["hits"])
```

Step 5: Write the orchestration script

run_pipeline.sh stops at the first failure and logs every stage with a UTC timestamp. When SCAN_URL is set it calls the deployed function; otherwise it runs the same handler through the local harness.

BASH

```
#!/usr/bin/env bash
# Capstone orchestrator: stops at the first failure and logs each
stage with a UTC timestamp.
set -euo pipefail
cd "$(dirname "$0")"
mkdir -p data out
log() { echo "$(date -u +%FT%TZ) $*" | tee -a out/pipeline.log; }

log "[1/4] Filtering raw logs"
python3 scripts/log_filter_cli.py --input data/raw.csv --output
data/filtered.csv --level ERROR

log "[2/4] Scanning for IOC matches"
if [ -n "${SCAN_URL:-}" ]; then                        # production: the
faasd function, e.g. http://host.docker.internal:8080/function/ioc-
alert
  curl -fsS -X POST "$SCAN_URL" --data-binary @data/filtered.csv >
data/scan_result.json
else                                                   # no faasd
available: run the same handler through the local harness
  node faas/harness.js faas/handler.js "$(cat data/filtered.csv)" >
data/scan_result.json
fi

log "[3/4] Sending alert if matches found"
python3 scripts/alert_from_scan.py data/scan_result.json

log "[4/4] Archiving results"
ARCHIVE="out/archive_$(date -u +%Y%m%d_%H%M%S).tar.gz"
tar -czf "$ARCHIVE" data/
sha256sum "$ARCHIVE" >> out/SHA256SUMS
log "Pipeline complete: $ARCHIVE"
```

Step 6: Run an end-to-end test and capture evidence

BASH

```
chmod +x run_pipeline.sh
./run_pipeline.sh
cat data/scan_result.json
sha256sum -c out/SHA256SUMS
```

Expected output (times differ): four stages are logged, the filter writes 2 rows with level=ERROR, the alert script prints Alert sent: ['203.0.113.99'], and sha256sum reports the archive as OK.

```text
2026-10-09T10:53:38Z [1/4] Filtering raw logs
2 rows with level=ERROR written to data/filtered.csv (0 skipped)
2026-10-09T10:53:38Z [2/4] Scanning for IOC matches
2026-10-09T10:53:38Z [3/4] Sending alert if matches found
Alert sent: ['203.0.113.99']
2026-10-09T10:53:38Z [4/4] Archiving results
2026-10-09T10:53:38Z Pipeline complete:
out/archive_20261009_105338.tar.gz
```

Step 7: Containerise the pipeline

Inside a container 127.0.0.1 means the container itself, so the function and the mail server on the host are reached through host.docker.internal. The slim image has no curl, so it is installed, and the pipeline runs as a non-root user. Only the out folder is mounted.

```dockerfile
FROM python:3.12-slim
RUN apt-get update && apt-get install -y --no-install-recommends curl
\
 && rm -rf /var/lib/apt/lists/*
WORKDIR /app
COPY . .
RUN useradd --create-home appuser && mkdir -p data out && chown -R
appuser /app
USER appuser
ENTRYPOINT ["./run_pipeline.sh"]
```

```bash
docker build -f docker/Dockerfile -t forensic-pipeline:1.0 .
docker run --rm --add-host=host.docker.internal:host-gateway \
  -e SCAN_URL=http://host.docker.internal:8080/function/ioc-alert \
  -e MAIL_HOST=host.docker.internal \
  -v "$PWD/out:/app/out" forensic-pipeline:1.0
```

Push the image to the local registry as forensic-pipeline:1.0 (docker tag, then docker push localhost:5000/forensic-pipeline:1.0).

Step 8: Wire it into CI/CD

Reuse the Lab 15 pipeline with an added Deploy stage (Jenkins) or job (GitHub Actions) that builds the new pipeline image and runs it on every successful build.

Step 9: Prepare the demo

Be ready to show: the pipeline diagram, a live or recorded run, the CI/CD build history, the alert e-mail in the SMTP sink, and your answer to “what happens if the scan stage fails?” (the script stops because of set -e).

Required Outcomes / Tasks

- Complete capstone repository (scripts/, faas/, docker/, jenkins/, run_pipeline.sh, and the design diagram).

- A successful end-to-end run log (or screen recording, system clock visible) from ingestion through to the archived output, its verified SHA-256, and the alert email.
- 5–10 minute live or recorded demo presented to the instructor, with a short Q&A on design trade-offs.
- One-page written reflection: which lab's tool was hardest to integrate, and how was the problem resolved?

Part II: CCSP-5213: Scripting Program and Cloud Architecture

Program: BS Cloud Applications Development and Operations

This part contains 16 sequential laboratory sessions (Lab 1 – Lab 16) aligned to the weekly theory schedule of DFSP-5208. Every lab uses only free and open-source tools and platforms (minimum versions are stated in each lab); no paid cloud accounts, licenses, or subscriptions are required to complete any exercise in this part.

Lab 1: Introduction to Python Scripting: Writing and Executing Basic Scripts

Course: CCSP-5213 | Scripting Program and Cloud Architecture | Week 1 | Program: BS Cloud Applications Development and Operations

Title

Introduction to Python Scripting: Writing and Executing Basic Scripts

Objectives

- Install and verify a modern Python 3 toolchain using only free, open-source components.
- Understand how scripting differs from compiled programming and why it matters for cloud automation.
- Write, save, and execute a first Python script from the command line and from VS Code.
- Use a virtual environment to keep project dependencies isolated and reproducible.

Preamble / Short Theory

Scripting languages such as Python are interpreted, high-level languages optimized for quickly automating repetitive tasks rather than building large compiled systems. In a cloud operations team that must manage resources, automate deployments, and monitor distributed application workloads, analysts rarely have time to write and compile a full application every time a repetitive task appears, instead they write a short script, run it immediately, and iterate. This lab establishes the toolchain (Python 3, a code editor, and a virtual environment) that will be reused throughout the semester, and introduces the read-write-execute cycle that underlies every later lab.

Prerequisites

- A laptop/desktop running Ubuntu or Kubuntu 24.04 LTS or newer, macOS, or Windows 11 with WSL2 (all free to use; Ubuntu is fully open source). Windows students complete every lab inside WSL2 with systemd enabled (see How to Use This Manual).
- Administrative rights to install software.
- Internet access for the initial download only (all tools work offline afterwards).
- No prior programming experience required.

Step-by-Step Guide

Step 1: Install Python 3.12 or newer (PSF open-source license)

Download the installer from the official Python website (python.org/downloads) or, on Ubuntu, install directly from the repositories. Ubuntu 24.04 provides Python 3.12, which is sufficient for every lab; Python 3.13 or 3.14 is equally fine if your lab PC already has it. Windows/macOS users must tick “Add Python to PATH” during installation.

BASH

```
# Ubuntu / Kubuntu / WSL2
sudo apt update
sudo apt install -y python3 python3-venv python3-pip
python3 --version   # confirm 3.12 or newer
```

Step 2: Install Visual Studio Code (free) or VSCodium (fully open source)

VS Code is free to use; students who prefer a build with no telemetry and a fully open-source license should install VSCodium instead both use the same editor engine and Python extension. Windows students install VS Code on Windows and add the WSL extension so the editor opens folders inside WSL2.

BASH

```
# Ubuntu (VS Code via snap) - or install the .deb from
code.visualstudio.com
sudo snap install code --classic
# Fully open-source alternative:
# sudo snap install codium --classic
```

Step 3: Create a project folder and virtual environment

A virtual environment keeps this course's Python packages separate from the rest of the system.

BASH

```
mkdir -p ~/cloud-scripting/lab01 && cd ~/cloud-scripting/lab01
python3 -m venv .venv
source .venv/bin/activate        # Windows PowerShell (outside
WSL): .venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
```

Step 4: Write the first script

In VS Code, create a file named hello_cloud.py with the following content. It prints the environment, a UTC timestamp and the SHA-256 hash of the script itself, which introduces the integrity idea used all semester.

PYTHON

```
import hashlib, pathlib, platform, datetime

ROLL = "YOUR-ROLL-NUMBER"   # replace with your own roll number
print("Scripting environment ready.")
print("Student:", ROLL)
print(f"Python {platform.python_version()} running on
{platform.system()}")
print("Course: CCSP-5213 - Scripting Program and Cloud Architecture")
print("UTC:",
datetime.datetime.now(datetime.timezone.utc).isoformat(timespec="secon
ds"))
print("SHA256 of this script:",
hashlib.sha256(pathlib.Path(__file__).read_bytes()).hexdigest())
```

Step 5: Execute the script from the terminal

Run the script and observe the interpreted, line-by-line execution.

BASH

```
python hello_cloud.py
```

Step 6: Run the same script inside VS Code

Open the folder in VS Code (code .), install the Python extension when prompted, select the .venv interpreter (bottom-right status bar), then use Run ▸ Run Without Debugging.

Step 7: Install Git and Docker once (used from Lab 3 and Lab 8 onward)

Install both tools now so later labs do not have to repeat the set-up. Docker is installed from the Ubuntu repositories, so no unreviewed script is piped into a root shell. Log out and back in (or run newgrp docker) after adding yourself to the docker group.

BASH

```
sudo apt install -y git docker.io
sudo systemctl enable --now docker
sudo usermod -aG docker $USER && newgrp docker
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git --version && docker --version
docker run --rm hello-world
```

Podman (sudo apt install podman) is a fully open-source, daemon-less alternative; Docker's own apt repository (docs.docker.com) can be used instead of docker.io if your instructor prefers the upstream package.

Step 8: Exclude the virtual environment from Git

Create a .gitignore file so that the virtual environment, caches, secrets and evidence files are never committed, then make the first commit.

TEXT

```
.venv/
__pycache__/
.env
*.log
evidence/
```

BASH

```
git init
git add .gitignore hello_cloud.py
git commit -m "Lab 1: first script and .gitignore"
git status    # .venv must not be listed
```

Required Outcomes / Tasks

- Screenshot of `python3 --version` showing 3.12 or newer, with the system clock visible.
- Submit hello_cloud.py (with your own roll number filled in) together with a terminal screenshot of its output, system clock visible.
- One-paragraph written reflection: how does an interpreted script differ from a compiled C/Java program?
- Screenshot of `git status` showing that the virtual environment (.venv) folder is excluded by .gitignore (used from Lab 3 onward).
- Screenshot of `docker run --rm hello-world` succeeding (or a note from your instructor that Docker is not available on your lab PC).

Lab 2: File Handling and Automation in Python

Course: CCSP-5213 | Scripting Program and Cloud Architecture | Week 2 | Program: BS Cloud Applications Development and Operations

Title

File Handling and Automation in Python

Objectives

- Read, write, and append text and CSV files using Python's standard library only.
- Automate a repetitive cloud data-processing task without manual copy-pasting.
- Apply context managers (with) to guarantee files are closed safely.

Preamble / Short Theory

Most real cloud automation tasks begin and end with a file: a log dump, a CSV export, or a folder of evidence files that must be renamed or summarized. Python's built-in `open()`, the `csv` module, and the `pathlib` module (all part of the free standard library, no extra installation needed) are sufficient for the overwhelming majority of file-handling tasks and are the foundation for the log-parsing and API labs later in the course.

Programs also fail: a file may be missing or a row malformed. A `try/except` block lets a script report the problem clearly instead of crashing; this lab uses it once, and Lab 11 develops the idea fully.

Prerequisites

- Lab 1 completed (Python 3.12 or newer + venv working).
- A sample CSV file (students may generate one with the script in Step 2 if none is supplied).

Step-by-Step Guide

Step 1: Activate the environment and create a working folder

BASH

```
cd ~/cloud-scripting
mkdir lab02 && cd lab02
source ../lab01/.venv/bin/activate
```

Step 2: Generate a sample dataset

Create generate_sample.py to produce a synthetic cloud billing/usage dataset for practice (no real/sensitive data is used). The fixed random seed means every student gets the same data, so results can be checked: with seed 42 you should see 12 ERROR, 16 WARN and 22 INFO rows.

PYTHON

```
import csv, random, datetime

random.seed(42)                          # same data on every run, so
answers can be checked
now = datetime.datetime.now(datetime.timezone.utc)
rows = []
for i in range(50):
    ts = now - datetime.timedelta(minutes=i * 7)
    rows.append([ts.isoformat(timespec="seconds"), f"host-
{random.randint(1, 5)}",
                 random.choice(["INFO", "WARN", "ERROR"]), f"event id
{i}"])
with open("sample_data.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerow(["timestamp", "host", "level", "message"])
    writer.writerows(rows)
print("sample_data.csv generated")
```

Step 3: Generate the resource inventory used by Assignment 1

Cloud work also starts from inventories. Create generate_resources.py to produce resources.csv with the columns id, region and status.

PYTHON

```
import csv, random

random.seed(42)
regions = ["us-east-1", "eu-west-1", "ap-south-1"]
statuses = ["running", "stopped", "degraded"]
with open("resources.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerow(["id", "region", "status"])
    for i in range(1, 31):
        writer.writerow([f"res-{i:03d}", random.choice(regions),
random.choice(statuses)])
print("resources.csv generated")
```

Step 4: Read, filter and count the file

Write filter_errors.py to extract only ERROR-level rows using the csv module and pathlib, count every level, and append a UTC run-log entry (append, not overwrite) each time the script executes.

PYTHON

```
import csv, datetime
from pathlib import Path

src, dst = Path("sample_data.csv"), Path("errors_only.csv")
counts = {"INFO": 0, "WARN": 0, "ERROR": 0}
with src.open(newline="", encoding="utf-8") as inp, dst.open("w",
newline="", encoding="utf-8") as out:
    reader = csv.DictReader(inp)
    writer = csv.DictWriter(out, fieldnames=reader.fieldnames)
    writer.writeheader()
    for row in reader:
        counts[row["level"]] = counts.get(row["level"], 0) + 1
        if row["level"] == "ERROR":
            writer.writerow(row)
print(f"{counts['ERROR']} ERROR rows written to {dst};
{counts['WARN']} WARN rows seen; {counts['INFO']} INFO rows seen")

with open("run_history.log", "a", encoding="utf-8") as log:

log.write(f"{datetime.datetime.now(datetime.timezone.utc).isoformat(ti
mespec='seconds')} - filter_errors.py executed\n")
```

Step 5: Handle a missing file (exception primer)

Wrap a risky step in try/except and exit with a clear message instead of a traceback.

PYTHON

```
from pathlib import Path

src = Path("sample_data.csv")
try:
    text = src.read_text(encoding="utf-8")
except FileNotFoundError:
    raise SystemExit(f"Input file not found: {src}")
print(f"{len(text.splitlines())} lines read from {src}")
```

Step 6: Execute and verify

BASH

```
python generate_sample.py
python filter_errors.py
head errors_only.csv
cat run_history.log
```

Required Outcomes / Tasks

- sample_data.csv, errors_only.csv, and run_history.log submitted together, with a screenshot of the run (system clock visible).
- resources.csv generated with generate_resources.py (30 rows; id, region, status).

- Extend filter_errors.py so it also writes warn_only.csv and prints the percentage of rows at each level; submit the modified script.
- Explain in 2–3 sentences why `with open(...)` is preferred over calling `open()/close()` manually.

Lab 3: Command-Line Tools with Python (argparse)

Course: CCSP-5213 | Scripting Program and Cloud Architecture | Week 3 | Program: BS Cloud Applications Development and Operations

Title

Command-Line Tools with Python (argparse)

Objectives

- Build a reusable command-line utility using the free, built-in `argparse` module.
- Accept input file paths, flags, and options from the terminal instead of hard-coding them.
- Package the tool so classmates can run it with `--help` to see usage instructions.

Preamble / Short Theory

Hard-coded scripts (Lab 2) are useful once; command-line tools are reusable. `argparse`, included with every Python installation at no cost, turns a script into a small program with named options, default values, and automatic `--help` text, the same pattern used by professional cloud tooling. This lab converts the CSV filter built in Lab 2 into a flexible CLI utility.

Real tools also report what they do. Python's `logging` module (standard library) writes timestamped messages with a severity level, which is far more useful than print statements; Lab 12 returns to it in depth. This lab also begins the Git habit: every lab from now on ends with a commit.

Prerequisites

- Lab 2 completed.
- sample_data.csv from Lab 2 (or regenerate it).
- Git installed and configured (Lab 1, Step 7).

Step-by-Step Guide

Step 1: Create the CLI script

Create log_filter_cli.py accepting --input, --output, --level and --limit arguments. The corrected version counts the rows it writes in a plain loop; an earlier draft printed “0 rows written” because writer.writerow() returns the number of characters written, not None.

PYTHON

```
#!/usr/bin/env python3
"""Filter a CSV of log records by level. Usage: log_filter_cli.py --
input IN --output OUT [--level L] [--limit N]"""
import argparse, csv, logging, sys
from pathlib import Path

logging.basicConfig(level=logging.INFO,
format="%(asctime)s %(levelname)s %(message)s")


def main():
    parser = argparse.ArgumentParser(description="Filter cloud CSV
records by log level.")
    parser.add_argument("--input", required=True, help="Path to source
CSV")
    parser.add_argument("--output", required=True, help="Path to write
filtered CSV")
    parser.add_argument("--level", default="ERROR", choices=["INFO",
"WARN", "ERROR"], help="Level to keep")
    parser.add_argument("--limit", type=int, default=0, help="Stop
after N matching rows (0 = no limit)")
    args = parser.parse_args()

    src, dst = Path(args.input), Path(args.output)
    if not src.exists():
        logging.error("Input file not found: %s", src)
        return 1
    dst.parent.mkdir(parents=True, exist_ok=True)

    written = skipped = 0
    with src.open(newline="", encoding="utf-8") as inp, dst.open("w",
newline="", encoding="utf-8") as out:
        reader = csv.DictReader(inp)
        writer = csv.DictWriter(out, fieldnames=reader.fieldnames)
        writer.writeheader()
        for line_no, row in enumerate(reader, start=2):       # line 1
is the header
            level = row.get("level")
            if level is None:
                skipped += 1
                logging.warning("line %d skipped: no 'level' column
value", line_no)
                continue
            if level == args.level:
                writer.writerow(row)
                written += 1
                if args.limit and written >= args.limit:
                    break
    print(f"{written} rows with level={args.level} written to {dst}
({skipped} skipped)")
```

```
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

Step 2: Run with --help

BASH

```
python log_filter_cli.py --help
```

Step 3: Run against the sample data

BASH

```
python log_filter_cli.py --input ../lab02/sample_data.csv --output
warn_only.csv --level WARN
python log_filter_cli.py --input ../lab02/sample_data.csv --output
out/warn3.csv --level WARN --limit 3
python log_filter_cli.py --input nope.csv --output x.csv; echo "exit
code: $?"   # message and exit code 1
```

The count printed on the last line must equal the number of data rows in the output file (wc -l minus the header line).

Step 4: Make the script executable (Linux/macOS)

The script already starts with the shebang line #!/usr/bin/env python3. Mark the file executable so it can be called directly, similar to a native command-line tool.

BASH

```
chmod +x log_filter_cli.py
./log_filter_cli.py --input ../lab02/sample_data.csv --output
warn_only.csv --level WARN
```

Step 5: Document the extra flag and commit

Document the --limit option in a short README.md and commit the lab to Git.

TEXT

```
# log_filter_cli
Filters a CSV of log records by level.

    python3 log_filter_cli.py --input IN.csv --output OUT.csv [--level
INFO|WARN|ERROR] [--limit N]

--limit N   stop after N matching rows (default 0 = no limit)
```

```bash
cd ~/cloud-scripting/lab03
git init
git add log_filter_cli.py README.md
git commit -m "Lab 3: argparse CLI with --limit"
```

Required Outcomes / Tasks

- log_filter_cli.py submitted with a screenshot of the --help output (system clock visible).
- Output CSV files for both ERROR and WARN levels, and the printed count matching wc `-l` minus one.
- README.md documenting --limit, plus one additional optional flag of your own choice (e.g. --host NAME).
- Screenshot of `git log --oneline` showing at least one commit for this lab.

Lab 4: Fundamentals of Bash Scripting

Course: CCSP-5213 | Scripting Program and Cloud Architecture | Week 4 | Program: BS Cloud Applications Development and Operations

Title

Fundamentals of Bash Scripting

Objectives

- Write and run Bash scripts on Linux (native, WSL2, or macOS Terminal, all free).
- Use variables, conditionals, and loops to automate directory operations.
- Understand exit codes and basic error handling in shell scripts.

Preamble / Short Theory

Bash (Bourne Again SHell) ships free with every Linux distribution and macOS, and is available on Windows through WSL2 (Windows Subsystem for Linux, free and open source). Much of cloud tooling is glued together with shell scripts because Bash has direct, low-overhead access to the filesystem and system utilities. This lab covers the syntax needed for the automation labs that follow.

Prerequisites

- A Bash shell: native on Linux/macOS, or WSL2 (`wsl --install`) on Windows 11.
- Basic familiarity with terminal navigation (cd, ls, mkdir).

Step-by-Step Guide

Step 1: Verify Bash version

BASH

```
bash --version   # GNU bash, version 5.x expected
```

Step 2: Create the lab folder

BASH

```
mkdir -p ~/cloud-scripting/lab04 && cd ~/cloud-scripting/lab04
```

Step 3: Write a directory-organiser script

Create organize.sh that creates dated sub-folders and moves files into them based on extension. The corrected script skips itself, counts items per category, handles names with spaces, and never overwrites an existing file (it prefixes the time instead, so both copies of evidence are kept).

BASH

```
#!/usr/bin/env bash
# organize.sh: move files in a folder into dated archive sub-folders
by extension.
# set -e: exit on any error; set -u: error on unset variables; set -o
pipefail: a pipeline fails if any stage fails.
set -euo pipefail

TARGET_DIR="${1:-.}"
[ -d "$TARGET_DIR" ] || { echo "Not a directory: $TARGET_DIR" >&2;
exit 1; }
DATE_TAG=$(date +%Y-%m-%d)
ARCHIVE="${TARGET_DIR}/archive_${DATE_TAG}"
SELF="$(basename "${BASH_SOURCE[0]}")"

mkdir -p "${ARCHIVE}/logs" "${ARCHIVE}/csv" "${ARCHIVE}/other"
n_logs=0; n_csv=0; n_other=0

move_safe() {            # never overwrite: on a name clash, prefix
the time so both copies are kept
  local src="$1" dest_dir="$2" base target
  base="$(basename "$src")"; target="${dest_dir}/${base}"
  [ -e "$target" ] && target="${dest_dir}/$(date +%H%M%S)_${base}"
  mv -- "$src" "$target"
}

for f in "${TARGET_DIR}"/*; do
  [ -f "$f" ] || continue
  [ "$(basename "$f")" = "$SELF" ] && continue          # never move
this script itself
  case "$f" in
    *.log) move_safe "$f" "${ARCHIVE}/logs";   n_logs=$((n_logs +
1)) ;;
    *.csv) move_safe "$f" "${ARCHIVE}/csv";     n_csv=$((n_csv +
1)) ;;
    *)     move_safe "$f" "${ARCHIVE}/other"; n_other=$((n_other +
1)) ;;
  esac
done

echo "Organised into ${ARCHIVE}: logs=${n_logs} csv=${n_csv}
other=${n_other}"
```

Step 4: Make it executable and test it

BASH

```
chmod +x organize.sh
touch test1.log test2.log "my file.log" test3.csv notes.txt
./organize.sh .
ls -R archive_*
```

Expected summary line: Organized into ./archive_YYYY-MM-DD: logs=3 csv=1 other=1. The script itself (organize.sh) must not have been moved.

Now test the name-clash protection: create another file called test1.log and run ./organize.sh . again. Both copies must survive, the new one with a time prefix.

Step 5: Explain the error handling

The comments in the script explain `set -euo pipefail`: exit on error (-e), exit on an unset variable (-u), and fail a pipeline if any stage fails (pipefail). Read them, then add one comment of your own describing what the move_safe function protects against.

Required Outcomes / Tasks

- organize.sh submitted along with a terminal screenshot of a successful run (system clock visible), showing the per-category counts.
- Extend the script with a fourth category (for example *.json into a json/ folder) and update the summary line; submit the modified script.
- Short written answer: what happens if `set -euo pipefail` is removed and the script hits an error midway? Give one example for each flag.

Lab 5: Automating System Tasks with Bash (cron)

Course: CCSP-5213 | Scripting Program and Cloud Architecture | Week 5 | Program: BS Cloud Applications Development and Operations

Title

Automating System Tasks with Bash (cron)

Objectives

- Schedule a Bash script to run automatically using cron (free, built into Linux).
- Automate a timestamped backup of cloud working files.
- Read and interpret crontab syntax and system logs for scheduled jobs.

Preamble / Short Theory

Manually running backup or cleanup scripts is unreliable; cron a scheduler bundled with every Linux distribution at no cost, runs scripts unattended at fixed intervals. This is the same mechanism production servers use to run nightly backups, log rotation, and health checks. This lab builds a backup script and schedules it, mirroring how a cloud operations team would automate evidence/data preservation.

Every command also ends with an exit code: 0 means success and anything else means failure. Cron cannot show a terminal, so a scheduled script must report problems through its exit code and a log file. The backup script below checks its own archive and records a SHA-256 hash, so an altered or damaged backup is detected.

Prerequisites

- Lab 4 completed.
- Access to a Linux shell with cron installed and systemd running (`sudo apt install cron` on Ubuntu if missing). On WSL, enable systemd as described in How to Use This Manual.

Step-by-Step Guide

Step 1: Confirm cron is installed and running

BASH

```
sudo apt install -y cron
sudo systemctl enable --now cron
sudo systemctl status cron --no-pager
```

Step 2: Write the backup script

Create the lab folder and write backup.sh. The corrected script excludes .venv and node_modules (otherwise every run archives hundreds of megabytes), writes a SHA-256 checksum, verifies that the archive can be read, and deletes backups older than 7 days.

BASH

```
mkdir -p ~/cloud-scripting/lab05 && cd ~/cloud-scripting/lab05
```

BASH

```
#!/usr/bin/env bash
# backup.sh: timestamped, verified backup with 7-day retention. Paths
can be overridden for testing.
set -euo pipefail
SRC_DIR="${SRC_DIR:-$HOME/cloud-scripting}"
BACKUP_ROOT="${BACKUP_ROOT:-$HOME/cloud-backups}"
TS=$(date -u +%Y%m%d_%H%M%S)
ARCHIVE="${BACKUP_ROOT}/backup_${TS}.tar.gz"
mkdir -p "$BACKUP_ROOT"

tar --exclude='.venv' --exclude='node_modules' -czf "$ARCHIVE" -C
"$SRC_DIR" .
( cd "$BACKUP_ROOT" && sha256sum "$(basename "$ARCHIVE")" >>
SHA256SUMS )
tar -tzf "$ARCHIVE" > /dev/null                        # verify the
archive can be read
echo "$(date -u +%FT%TZ) backup_${TS}.tar.gz created and verified" >>
"$BACKUP_ROOT/backup_history.log"

find "$BACKUP_ROOT" -name 'backup_*.tar.gz' -mtime +7 -delete   #
retention: 7 days
```

Step 3: Make it executable and test manually

BASH

```
chmod +x ~/cloud-scripting/lab05/backup.sh
mkdir -p ~/cloud-scripting/.venv && echo test > ~/cloud-
scripting/.venv/junk   # a file that must NOT be backed up
~/cloud-scripting/lab05/backup.sh
echo "exit code: $?"
ls ~/cloud-backups
cd ~/cloud-backups && sha256sum -c SHA256SUMS
tar -tzf backup_*.tar.gz | grep -c venv   # expect 0
```

A second test shows a failure being reported: run SRC_DIR=/nonexistent ~/cloud-scripting/lab05/backup.sh; echo $? and confirm that the exit code is not 0.

Step 4: Schedule it with cron

Edit the crontab to run the backup every 10 minutes for demonstration purposes (a real deployment would use a nightly schedule). The output redirection matters: cron has no terminal, so any error message would otherwise be lost.

BASH

```
crontab -e
# add these lines, then save:
# demonstration: every 10 minutes
*/10 * * * * $HOME/cloud-scripting/lab05/backup.sh >> $HOME/cloud-backups/cron.out 2>&1
# real use: nightly at 02:00
# 0 2 * * * $HOME/cloud-scripting/lab05/backup.sh >> $HOME/cloud-backups/cron.out 2>&1
```

Step 5: Verify scheduled execution

BASH

```
crontab -l
# wait 20+ minutes, then:
cat ~/cloud-backups/backup_history.log
```

On WSL without systemd, run sudo service cron start after each restart; Windows students without WSL may use Task Scheduler (schtasks).

Step 6: Copy the backup to S3-compatible storage with the cloud CLI

The course outline lists a cloud CLI. Use the free Moto S3 emulator so no cloud account is needed. The dummy credentials test/test are accepted by the emulator.

BASH

```
source ~/cloud-scripting/lab01/.venv/bin/activate
pip install "moto[server]" awscli
moto_server -p 5000 &
export AWS_ACCESS_KEY_ID=test AWS_SECRET_ACCESS_KEY=test AWS_DEFAULT_REGION=us-east-1
aws --endpoint-url http://localhost:5000 s3 mb s3://lab-backups
LATEST=$(ls -t ~/cloud-backups/backup_*.tar.gz | head -1)
aws --endpoint-url http://localhost:5000 s3 cp "$LATEST" s3://lab-backups/
aws --endpoint-url http://localhost:5000 s3 ls s3://lab-backups/
```

Required Outcomes / Tasks

- backup.sh submitted with the crontab entry shown in a screenshot (system clock visible).
- backup_history.log showing at least two automatic (unattended) runs about 10 minutes apart.
- Screenshot of `sha256sum -c SHA256SUMS` reporting OK, and of `grep -c venv` printing 0.
- Explain why the redirection `>> cron.out 2>&1` is needed in the crontab line, and what the retention line (`find ... -mtime +7 -delete`) does.
- Screenshot of the `aws s3 ls` listing showing the backup archive in the lab-backups bucket.

Lab 6: Web Scraping with Python (BeautifulSoup)

Course: CCSP-5213 | Scripting Program and Cloud Architecture | Week 6 | Program: BS Cloud Applications Development and Operations

Title

Web Scraping with Python (BeautifulSoup)

Objectives

- Install and use BeautifulSoup4 and requests — both free, open-source Python libraries.
- Extract structured data from a public web page and store it as CSV.
- Respect robots.txt and apply polite scraping practices (rate limiting, User-Agent headers).

Preamble / Short Theory

Web scraping automates the collection of publicly available web data in cloud work this ranges from gathering open-source threat intelligence to monitoring cloud-service status pages. `requests` fetches the raw HTML and `BeautifulSoup4` (both free/open source, installed via pip) parses it into a navigable tree so specific elements (tables, links, headlines) can be extracted programmatically instead of read by eye.

When a cloud provider's status page offers an RSS or JSON feed, prefer the feed over scraping its HTML: it is more stable and puts less load on the server.

Prerequisites

- Labs 1–3 completed (Python + venv working).
- Internet access to reach a public test page (or use the local test page in Step 5 if the site is unreachable).

Step-by-Step Guide

Step 1: Install the libraries (free, from PyPI)

BASH

```
cd ~/cloud-scripting && mkdir lab06 && cd lab06
source ../lab01/.venv/bin/activate
pip install requests beautifulsoup4
```

Step 2: Check the target site's scraping policy

Before scraping any site, open its /robots.txt and confirm the path you intend to fetch is not disallowed. For this lab we use https://books.toscrape.com, a site built specifically for scraping practice.

Step 3: Write the scraper

The scraper identifies itself with a User-Agent, sets a timeout, handles network errors with try/except, tolerates a malformed item instead of crashing, and pauses between requests (a pause placed inside a loop over results that are already downloaded would not rate-limit anything).

PYTHON

```
import csv, sys, time
import requests
from bs4 import BeautifulSoup

URL = sys.argv[1] if len(sys.argv) > 1 else
"https://books.toscrape.com/"
HEADERS = {"User-Agent": "Lab06-Student-Scraper/1.0 (course
exercise)"}

try:
    resp = requests.get(URL, headers=HEADERS, timeout=10)
    resp.raise_for_status()
except requests.exceptions.Timeout:
    sys.exit(f"Timed out fetching {URL}")
except requests.exceptions.RequestException as exc:
    sys.exit(f"Request failed: {exc}")

soup = BeautifulSoup(resp.text, "html.parser")
rows = []
for book in soup.select("article.product_pod"):
    link = book.h3.a if book.h3 else None
    price = book.select_one(".price_color")
    if link is None or price is None:                    # tolerate a
missing element
        print("skipped one malformed item", file=sys.stderr)
        continue
    rows.append([link["title"], price.text.strip()])

with open("scraped_books.csv", "w", newline="", encoding="utf-8") as
f:
    writer = csv.writer(f)
    writer.writerow(["title", "price"])
    writer.writerows(rows)
print(f"Saved {len(rows)} records to scraped_books.csv")
time.sleep(1)                                             # one
request, then a polite pause before any next page
```

Step 4: Run and inspect the output

BASH

```
python scraper.py
head -5 scraped_books.csv
```

On the real site the first page returns 20 books.

Step 5: Test offline against a local page and a failure

Public sites are not always reachable. Create a tiny local page that contains one good item and one malformed item, serve it, and run the scraper against it. Then point the scraper at a port where nothing is listening to see the error handling.

BASH

```
mkdir -p testpage && cat > testpage/index.html <<'EOF'
<html><body>
<article class="product_pod"><h3><a title="Local Book One"
href="#">Local Book One</a></h3><p
class="price_color">&pound;10.00</p></article>
<article class="product_pod"><h3><a title="Broken Item"
href="#">Broken Item</a></h3></article>
</body></html>
EOF
python3 -m http.server 8765 --directory testpage &
python scraper.py http://127.0.0.1:8765/     # skipped one malformed
item, Saved 1 records
python scraper.py http://127.0.0.1:1/ ; echo "exit code: $?"   #
Request failed ..., exit code 1
kill %1
```

The same code works for a status or threat-intelligence page: change the URL and the CSS selectors, and prefer a feed when one exists.

Required Outcomes / Tasks

- scraper.py and scraped_books.csv submitted, with a screenshot of the run (system clock visible).
- Screenshot of the offline test (one record saved, one malformed item skipped) and of the failure case with its exit code.
- Extend the scraper to follow the “next” page link for up to 3 pages, with a 1-second pause between page requests.
- One paragraph explaining what robots.txt is and why rate limiting matters ethically and legally.

Lab 7: Node.js Scripting for File System Automation

Course: CCSP-5213 | Scripting Program and Cloud Architecture | Week 7 | Program: BS Cloud Applications Development and Operations

Title

Node.js Scripting for File System Automation

Objectives

- Install a current Node.js LTS release (22 or 24; free, open source, OpenJS Foundation) and verify npm.
- Use Node's built-in `fs` and `path` modules to automate file renaming safely (dry run first, never overwrite).
- Compare JavaScript/Node.js scripting patterns with the Python equivalents from Lab 2.

Preamble / Short Theory

Node.js lets JavaScript run outside the browser, and is entirely free and open source. Many cloud-native and forensic tooling ecosystems (including serverless platforms used later in the course) run on Node.js, so this lab introduces its asynchronous file-handling model using the built-in `fs/promises` API, no paid packages required.

Prerequisites

- Administrative rights to install Node.js (current LTS: 22 or 24).
- Comfort with basic JavaScript syntax (variables, functions) is helpful but not required.
- Git installed and configured (Lab 1, Step 7).

Step-by-Step Guide

Step 1: Install the current Node.js LTS (22 or 24)

Do not pipe a downloaded script straight into a root shell. Use the snap package, or download the NodeSource set-up script, read it, and only then run it.

BASH

```
# Option A (simplest on Ubuntu/Kubuntu)
sudo snap install node --classic --channel=24

# Option B: download, inspect, then run
curl -fsSLo nodesource_setup.sh https://deb.nodesource.com/setup_24.x
less nodesource_setup.sh          # read it before running it
sudo -E bash nodesource_setup.sh
sudo apt install -y nodejs

node --version   # v22.x or v24.x
npm --version
```

Step 2: Create the project

BASH

```
mkdir -p ~/cloud-scripting/lab07 && cd ~/cloud-scripting/lab07
npm init -y
```

Step 3: Write a batch file-renamer

Create rename_cloud_files.js that adds a standardised timestamp prefix to every file in a folder. It is a dry run unless --apply is given, skips files that already carry a timestamp prefix, never overwrites, and exits with code 1 on an error.

JAVASCRIPT

```
// Usage: node rename_cloud_files.js <dir> [--apply]   (dry run by
default; never overwrites)
const fs = require('fs/promises');
const path = require('path');

const STAMPED = /^\d{4}-\d{2}-\d{2}T\d{2}-\d{2}-\d{2}-\d{3}Z_/;

async function renameFiles(dir, apply) {
  const stamp = new Date().toISOString().replace(/[:.]/g, '-');
  let count = 0, skipped = 0;
  for (const file of await fs.readdir(dir)) {
    const oldPath = path.join(dir, file);
    if (!(await fs.stat(oldPath)).isFile()) continue;
    if (STAMPED.test(file)) { skipped++; continue; }              //
already has a timestamp prefix
    const newPath = path.join(dir, `${stamp}_${file}`);
    try { await fs.access(newPath); console.error(`refusing to
overwrite ${newPath}`); continue; } catch {}
    console.log(`${apply ? 'RENAME' : 'DRY RUN'} ${file} ->
${path.basename(newPath)}`);
    if (apply) await fs.rename(oldPath, newPath);
    count++;
  }
  console.log(`${apply ? 'Renamed' : 'Would rename'} ${count} files,
skipped ${skipped}, in ${dir}`);
}

const args = process.argv.slice(2);
renameFiles(args.find(a => !a.startsWith('--')) || '.',
args.includes('--apply'))
  .catch(err => { console.error('Error:', err.message);
process.exit(1); });
```

Step 4: Test the script

BASH

```
mkdir sample_files && touch sample_files/a.txt sample_files/b.csv
node rename_cloud_files.js sample_files            # dry run: shows
what would change
node rename_cloud_files.js sample_files --apply    # renames the files
node rename_cloud_files.js sample_files --apply    # second run:
renames 0, skips 2
ls sample_files
node rename_cloud_files.js nodir; echo "exit code: $?"   # error
message and exit code 1
```

Step 5: Practise the Git branch and pull-request workflow

Commit the working script on the main branch, then make your change on a feature branch. Push the branch to GitHub (free tier) or Gitea and open a pull request; ask a classmate to review and approve it.

BASH

```
git init
git add rename_cloud_files.js package.json
git commit -m "Lab 7: dry-run renamer"
git switch -c feature/prefix-option
# make your change (see Required Outcomes), then:
git add -A && git commit -m "Add --prefix option"
```

Required Outcomes / Tasks

- rename_cloud_files.js submitted with before/after directory listing screenshots (system clock visible), including the dry run.
- Extend the script with a `--prefix NAME` option that replaces the timestamp text in the new file name, on a feature branch, and submit the pull-request link or a screenshot of the reviewed pull request.
- Short comparison table (3 rows) contrasting Python's `pathlib`/`os` calls with Node's `fs/promises` equivalents.

Lab 8: Deploying Node.js Scripts as Serverless Functions

Course: CCSP-5213 | Scripting Program and Cloud Architecture | Week 8 | Program: BS Cloud Applications Development and Operations

Title

Deploying Node.js Scripts as Serverless Functions

Objectives

- Understand the Functions-as-a-Service (FaaS) model without a paid cloud subscription.
- Write and test a Node.js cloud-usage-alert function locally with a small harness that mimics the OpenFaaS Node template (Tier 1, required).
- Optionally deploy the same function with OpenFaaS on faasd, invoke it over HTTP and inspect its logs (Tier 2, optional).

Preamble / Short Theory

Serverless computing runs code on demand without managing servers. Instead of a billed AWS Lambda/Azure Functions account, this lab first writes and tests the function locally with a small harness (Tier 1, required and graded) and then, optionally, deploys it with OpenFaaS on faasd (Tier 2), a lightweight FaaS engine that needs only Linux with systemd and containerd, with no Kubernetes cluster. Check the License before installing faasd: the Community Edition is licensed for non-commercial, personal use under the faasd CE EULA, so confirm with your department that classroom use is covered. The local harness has no such restriction. This is a realistic way to learn serverless deployment patterns for free before touching a billed cloud account.

Prerequisites

- Lab 7 completed (Node.js basics).
- Tier 1 needs only Node.js. Tier 2 (optional) also needs Docker (installed in Lab 1, Step 7), Linux with systemd, and faas-cli.

Step-by-Step Guide

Step 1: Create the lab folder and the local harness (Tier 1, required)

The harness loads a handler file and calls it the same way the OpenFaaS Node template does, so the function can be tested without faasd.

BASH

```
mkdir -p ~/cloud-scripting/lab08 && cd ~/cloud-scripting/lab08
```

JAVASCRIPT

```
// Local harness that mimics the OpenFaaS node template, so the
function can be tested without faasd.
const file = process.argv[2];
const handler = require(require('path').resolve(file));
const body = process.argv[3] ?? '';
const ctx = { status(code) { return { succeed: b => ({ status: code,
body: b }) }; } };
handler({ body }, ctx).then(r => console.log(JSON.stringify(r)));
```

Step 2: Write the function logic

Create handler.js. It is the same file you would later place in usage-alert/handler.js for faasd.

JAVASCRIPT

```
'use strict'
const THRESHOLD_USD = Number(process.env.THRESHOLD_USD || 75);   //
set to the instructor's value

module.exports = async (event, context) => {
  let payload;
  try { payload = JSON.parse(event.body || '{}'); }
  catch (e) { return context.status(400).succeed({ error: 'body must
be JSON' }); }
  const cost = Number(payload.costUsd);
  if (!payload.resource || Number.isNaN(cost)) {
    return context.status(400).succeed({ error: 'resource and numeric
costUsd are required' });
  }
  return context.status(200).succeed({ resource: payload.resource,
costUsd: cost, overBudget: cost > THRESHOLD_USD });
}
```

Step 3: Test the function

BASH

```
node harness.js handler.js '{"resource":"vm-01","costUsd":72.5}'
# under the default 75
THRESHOLD_USD=50 node harness.js handler.js '{"resource":"vm-
01","costUsd":72.5}'   # over a 50 threshold
node harness.js handler.js 'timestamp,host'
# not JSON: status 400
```

Expected: the first run reports "overBudget":false, the second "overBudget":true, and the third returns status 400 with "body must be JSON" instead of crashing.

Step 4: Understand “stateless”

A serverless function keeps nothing between calls: every invocation starts fresh, so anything that must persist (alerts already sent, counters) has to live in a database or file outside the function. Lab 10 returns to this idea.

Step 5: Check the License and prerequisites (Tier 2, optional)

Only continue if your department confirmed that the faasd CE EULA covers classroom use, and your machine runs Linux with systemd (not plain Windows; use WSL2 with systemd enabled). Everything above this step is enough for full marks.

Step 6: Install faasd and faas-cli (Tier 2)

Follow the faasd handbook for your faasd version; the commands below are the ones used when this manual was revised. The faas-cli installer is downloaded and read before it is run.

```bash
git clone https://github.com/openfaas/faasd --depth=1
cd faasd
sudo ./hack/install.sh
sudo systemctl status faasd   # should show active (running)
cd ..

curl -sSLo get-faas-cli.sh https://cli.openfaas.com
less get-faas-cli.sh          # read it before running it
sudo sh get-faas-cli.sh
faas-cli version
```

Step 7: Start a local registry and scaffold the function (Tier 2)

faasd has no built-in registry, and faas-cli up builds and pushes the image, so run a local registry first. List the available Node templates and use the newest one (the node20 template is the oldest you should use: Node 20 reached end of life in April 2026).

```bash
docker run -d -p 5000:5000 --name registry registry:2
export OPENFAAS_URL=http://127.0.0.1:8080
sudo cat /var/lib/faasd/secrets/basic-auth-password | faas-cli login --username admin --password-stdin
faas-cli template store list
faas-cli template store pull <newest-node-template>
faas-cli new usage-alert --lang <newest-node-template>
```

Step 8: Deploy, invoke and inspect logs (Tier 2)

Copy handler.js into usage-alert/handler.js and open usage-alert.yml. Change the image line to 127.0.0.1:5000/usage-alert:latest, then build, deploy and invoke. Check the faasd handbook for the registry and pull-policy settings of your faasd version if the pull fails.

```bash
cp handler.js usage-alert/handler.js
faas-cli up -f usage-alert.yml
curl -X POST http://127.0.0.1:8080/function/usage-alert -H "Content-Type: application/json" -d '{"resource":"vm-01","costUsd":72.5}'
faas-cli list
faas-cli logs usage-alert
```

Required Outcomes / Tasks

- Harness output for an over-budget and an under-budget resource, system clock visible.
- Change THRESHOLD_USD to an instructor-provided value and resubmit handler.js with new test output.
- Output showing that a non-JSON body is rejected with status 400 instead of crashing the function.
- Optional (Tier 2): screenshot of `faas-cli list` showing the deployed usage-alert function and the curl output.

Lab 9: REST API Interaction: Fetching and Parsing JSON Data

Course: CCSP-5213 | Scripting Program and Cloud Architecture | Week 10 | Program: BS Cloud Applications Development and Operations

Title

REST API Interaction: Fetching and Parsing JSON Data

Objectives

- Send GET requests to a REST API using Python `requests` and Node.js `fetch`/`axios` (free, OSS) and handle non-200 responses explicitly.
- Parse JSON responses relevant to cloud resource/billing data, and read cloud-state JSON from an S3 emulator.
- Preserve each response as timestamped evidence with a SHA-256 hash and a custody-log entry, then store parsed records in a local SQLite database file (in place of a paid cloud DB).

Preamble / Short Theory

Cloud operations tooling constantly polls REST APIs for resource inventories, billing summaries, and health checks. Python's `requests` and Node's `axios` are the standard free, open-source libraries for this. SQLite, bundled with Python at no cost, stands in for a managed cloud database (e.g., RDS/Cosmos DB) so the lab needs no billing account.

Operational records need the same discipline as evidence: save the raw response in a file whose name carries a UTC timestamp, record its SHA-256 hash, and append a line to a custody log. Never parse an error response as if it were data: check the status code first. Because a public to-do list is not a cloud service, the lab also reads real cloud-style state from the Moto emulator, which speaks the AWS API without an account.

Prerequisites

- Labs 1–3 and 7 completed.
- Internet access to a public test API (https://jsonplaceholder.typicode.com), or the local mock API in Step 2 if it is unreachable.

Step-by-Step Guide

Step 1: Set up the project

BASH

```
cd ~/cloud-scripting && mkdir lab09 && cd lab09
source ../lab01/.venv/bin/activate
pip install requests
sudo apt install -y sqlite3     # command-line client used in Step 5
```

Step 2: Start the local mock API (optional fallback)

If the public API is unreachable, create mock_api.py and run it in a second terminal. Then point the scripts at it with the API_URL environment variable; leave API_URL unset to use the public site.

PYTHON

```
# mock_api.py - a tiny local stand-in for the public test API (no
internet needed)
import json
from http.server import BaseHTTPRequestHandler, HTTPServer

POSTS = [{"userId": (i % 5) + 1, "id": i, "title": f"case note {i}",
"body": f"details for note {i}"}
         for i in range(1, 101)]
TODOS = [{"userId": (i % 5) + 1, "id": i, "title": f"resource task
{i}", "completed": i % 3 == 0}
         for i in range(1, 101)]

class Handler(BaseHTTPRequestHandler):
    def do_GET(self):
        data = {"/posts": POSTS, "/todos": TODOS}.get(self.path)
        code = 200 if data is not None else 404
        body = json.dumps(data if data is not None else {"error": "not
found"}).encode()
        self.send_response(code)
        self.send_header("Content-Type", "application/json")
        self.send_header("Content-Length", str(len(body)))
        self.end_headers()
        self.wfile.write(body)

    def log_message(self, *args):      # keep the terminal quiet
        pass

if __name__ == "__main__":
    print("Mock API listening on http://127.0.0.1:8765 (Ctrl+C to
stop)")
    HTTPServer(("127.0.0.1", 8765), Handler).serve_forever()
```

BASH

```
python mock_api.py &
export API_URL=http://127.0.0.1:8765
```

Step 3: Write fetch_resources.py (evidence workflow)

The script checks the status code before parsing, saves the raw response as a timestamped evidence file, hashes it, writes a custody-log line, and stores 20 parsed records with a parameterised SQL statement.

PYTHON

```
import datetime, hashlib, json, os, sqlite3, sys
import requests

BASE = os.getenv("API_URL", "https://jsonplaceholder.typicode.com")
URL = f"{BASE}/todos"


def utc_stamp():
    return
datetime.datetime.now(datetime.timezone.utc).strftime("%Y%m%d_%H%M%S")


def fetch(url):
    resp = requests.get(url, timeout=10)
    if resp.status_code != 200:                                    # non-
200: report and stop, never parse an error body
        raise RuntimeError(f"HTTP {resp.status_code} from {url}:
{resp.text[:120]}")
    return resp.json()


def save_evidence(data, url):
    ts = utc_stamp()
    fname = f"evidence_{ts}.json"
    with open(fname, "w", encoding="utf-8") as f:
        json.dump(data, f, indent=2)
    with open(fname, "rb") as f:
        digest = hashlib.sha256(f.read()).hexdigest()
    with open("custody.log", "a", encoding="utf-8") as log:
        log.write(f"{ts}\t{url}\t{fname}\tSHA256={digest}\n")
    return fname, digest


def store(data):
    conn = sqlite3.connect("resources.db")
    with conn:
        conn.execute("""CREATE TABLE IF NOT EXISTS tasks (
            id INTEGER PRIMARY KEY, user_id INTEGER, title TEXT,
completed INTEGER)""")
        for item in data[:20]:
            conn.execute("INSERT OR REPLACE INTO tasks VALUES
(?,?,?,?)",
                         (item["id"], item["userId"], item["title"],
int(item["completed"])))
    conn.close()
    return min(len(data), 20)


if __name__ == "__main__":
    try:
```

```
        data = fetch(URL)
        print(save_evidence(data, URL))
        print(f"{store(data)} records stored in resources.db")
    except requests.exceptions.RequestException as exc:
        sys.exit(f"Network failure: {exc}")
    except RuntimeError as exc:
        sys.exit(f"API failure: {exc}")
```

Step 4: Run it and verify the evidence

BASH

```
python fetch_resources.py
cat custody.log
sha256sum evidence_*.json     # must match the SHA256 value in
custody.log
```

Step 5: Query the stored data

BASH

```
sqlite3 resources.db "SELECT id, title, completed FROM tasks LIMIT 5;"
```

Step 6: Node.js equivalent with the built-in fetch

Node.js includes fetch, so no extra package is needed. Write fetch_resources.js:

JAVASCRIPT

```
const BASE = process.env.API_URL ||
'https://jsonplaceholder.typicode.com';

async function main() {
  try {
    const res = await fetch(`${BASE}/todos`, { signal:
AbortSignal.timeout(10000) });
    if (!res.ok) throw new Error(`HTTP ${res.status}`);          //
non-200 handled explicitly
    const data = await res.json();
    console.table(data.slice(0, 5).map(p => ({ id: p.id, title:
p.title })));
  } catch (err) {
    console.error('API error:', err.message);
    process.exitCode = 1;
  }
}
main();
```

If you prefer axios (free, OSS), install it with npm install axios and use this version, which also handles a non-200 status explicitly:

JAVASCRIPT

```
const url = (process.env.API_URL ||
'https://jsonplaceholder.typicode.com') + '/todos';
const axios = require('axios');
axios.get(url, { timeout: 10000, validateStatus: () => true })
  .then(res => {
    if (res.status !== 200) throw new Error(`HTTP ${res.status}`);
    console.table(res.data.slice(0, 5).map(p => ({ id: p.id, title:
p.title })));
  })
  .catch(err => { console.error('API error:', err.message);
process.exitCode = 1; });
```

Step 7: Test the failure paths

BASH

```
API_URL=http://127.0.0.1:8765/bad python fetch_resources.py; echo
"exit code: $?"   # HTTP 404 reported, exit code 1
API_URL=http://127.0.0.1:1 node fetch_resources.js; echo "exit code:
$?"   # fetch failed, exit code 1
```

Step 8: Read cloud state from an S3 emulator (Moto)

Start the Moto server, create a bucket with boto3, then save the emulator's cloud-state JSON as evidence with the same save_evidence function. (Check the data path on your Moto version if it differs.)

BASH

```
pip install "moto[server]" boto3
moto_server -p 5000 &
python3 - <<'PY'
import boto3
s3 = boto3.client("s3", endpoint_url="http://localhost:5000",
region_name="us-east-1",
                  aws_access_key_id="test",
aws_secret_access_key="test")
s3.create_bucket(Bucket="lab-evidence")
print("bucket created")
PY
curl -s http://localhost:5000/moto-api/data.json | head -c 400
```

PYTHON

```
import requests
from fetch_resources import save_evidence

URL = "http://localhost:5000/moto-api/data.json"
resp = requests.get(URL, timeout=10)
resp.raise_for_status()
print(save_evidence(resp.json(), URL))
```

Step 9: Keys and rate limits

Real APIs need a key and limit how fast you may call them. Never write a key into the code: keep it in a .env file that .gitignore excludes, load it into the environment, and send it in a header. If the server answers 429 (too many requests), wait for the time in the Retry-After header and try again.

BASH

```
# .env  (never commit this file)
API_KEY=replace-with-your-key

set -a; source .env; set +a      # load it into the environment
```

PYTHON

```
import os, time
API_KEY = os.getenv("API_KEY")
headers = {"Authorization": f"Bearer {API_KEY}"} if API_KEY else {}
resp = requests.get(URL, headers=headers, timeout=10)
if resp.status_code == 429:
    time.sleep(int(resp.headers.get("Retry-After", "5")))
    resp = requests.get(URL, headers=headers, timeout=10)
```

Required Outcomes / Tasks

- One evidence_*.json file, custody.log showing its matching SHA-256, and resources.db submitted, with a screenshot of the sqlite3 query (system clock visible).
- fetch_resources.js console.table output screenshot.
- Screenshots showing both scripts handling a non-200 status or a network failure without crashing, and exiting with a non-zero code.
- Evidence file and custody-log line created from the Moto emulator's cloud-state JSON.
- Add API-key loading from .env and 429 handling (Step 9) to the Python script, and document the behaviour in two sentences.

Lab 10: Automating Workflows with APIs

Course: CCSP-5213 | Scripting Program and Cloud Architecture | Week 11 | Program: BS Cloud Applications Development and Operations

Title

Automating Workflows with APIs

Objectives

- Chain a cloud-cost-check step with an automated email alert using only free, open-source components.
- Use a local SMTP test server (Mailpit, or Python's aiosmtpd) as the mail sink, with no real cloud mail service required.
- Make the workflow safe to schedule: remember what was already alerted in SQLite so the same resource is never e-mailed twice.

Preamble / Short Theory

Cost-overrun alerting is one of the most common cloud-automation workflows. A local SMTP sink lets `smtplib`/`nodemailer` be practised safely without real credentials or a paid mail relay: Mailpit shows captured messages in a web UI, and Python's aiosmtpd prints them in the terminal. (MailHog, used in earlier editions, has had no release since 2020.) This lab applies the same threshold idea as the Lab 8 usage-alert function to a list of resources in Python, mirroring real cloud cost-management alerting, and adds state so a scheduled job does not repeat itself.

Prerequisites

- Lab 8 completed (Tier 1 handler tested).
- Lab 9 completed (requests/fetch).
- Docker (installed in Lab 1) for Mailpit, or `pip install aiosmtpd` if Docker is not available.

Step-by-Step Guide

Step 1: Start a local SMTP sink (free, OSS)

Option A: Mailpit in Docker. Option B: Python's aiosmtpd, which prints each message in the terminal. Both listen on port 1025.

BASH

```
cd ~/cloud-scripting && mkdir lab10 && cd lab10
# Option A: Mailpit (web UI at http://localhost:8025)
docker run -d --name mailpit -p 1025:1025 -p 8025:8025 axllent/mailpit

# Option B: aiosmtpd (messages are printed in this terminal)
source ../lab01/.venv/bin/activate
pip install aiosmtpd
python -m aiosmtpd -n -l localhost:1025
```

Step 2: Write the alert workflow in Python

The script scans first, collects every match, and sends one consolidated email. If nothing matches it sends nothing; if the SMTP server is down it exits with a clear message.

PYTHON

```
import smtplib, sys
from email.mime.text import MIMEText

SMTP_HOST, SMTP_PORT = "localhost", 1025
BUDGET_USD = 50
RESOURCES = [("vm-01", 72.5), ("db-02", 20.0), ("vm-03", 130.0),
("cache-04", 49.99), ("vm-05", 51.0)]

def over_budget(resources):
    return [(name, cost) for name, cost in resources if cost >
BUDGET_USD]

def send_summary(findings):
    body = "Resources over budget\n" + "\n".join(f"- {name}:
${cost:.2f} (limit ${BUDGET_USD})" for name, cost in findings)
    msg = MIMEText(body)
    msg["Subject"] = f"Cloud Cost Alert: {len(findings)} resources
over budget"
    msg["From"], msg["To"] = "cost-automation@lab.local",
"ops@lab.local"
    with smtplib.SMTP(SMTP_HOST, SMTP_PORT, timeout=10) as server:
        server.send_message(msg)

if __name__ == "__main__":
    findings = over_budget(RESOURCES)
    if not findings:
        print("Within budget, no alert sent")
        sys.exit(0)
    try:
        send_summary(findings)                          # ONE consolidated
email, not one per resource
    except (OSError, smtplib.SMTPException) as exc:
        sys.exit(f"Alert failed: {exc}")
    print(f"Consolidated alert sent: {len(findings)} of
{len(RESOURCES)} resources over budget")
```

Step 3: Run the workflow and check the sink

BASH

```
python alert_workflow.py
# Mailpit: open http://localhost:8025 in a browser to view the
captured email
# aiosmtpd: the message appears in the terminal where it is running
```

Expected: one email listing vm-01, vm-03 and vm-05 (cost above 50), with the subject Cloud Cost Alert: 3 resources over budget.

Step 4: Node.js equivalent using nodemailer (free, OSS)

BASH

```
npm install nodemailer
```

Step 5: Write alert_workflow.js

JAVASCRIPT

```
const nodemailer = require('nodemailer');
const BUDGET_USD = 50;
const RESOURCES = [['vm-01', 72.5], ['db-02', 20.0], ['vm-03', 130.0],
['cache-04', 49.99], ['vm-05', 51.0]];
async function main() {
  const over = RESOURCES.filter(([, cost]) => cost > BUDGET_USD);
  if (!over.length) return console.log('Within budget, no alert
sent');
  const transporter = nodemailer.createTransport({ host: 'localhost',
port: 1025, secure: false });
  await transporter.sendMail({
    from: 'cost-automation@lab.local', to: 'ops@lab.local',
    subject: `Cloud Cost Alert: ${over.length} resources over budget`,
    text: over.map(([name, cost]) => `- ${name}: $${cost.toFixed(2)}
(limit $${BUDGET_USD})`).join('\n'),
  });
  console.log(`Consolidated alert sent: ${over.length} resources over
budget`);
}
main().catch(e => { console.error('Alert failed:', e.message);
process.exit(1); });
```

Step 6: Add state and a schedule

A scheduled workflow must not repeat itself. Keep an alerts table in SQLite and skip any entity that was alerted in the last 24 hours. Add the check before sending and an INSERT after a successful send, then schedule the script with cron.

PYTHON

```
import sqlite3, datetime
conn = sqlite3.connect("alerts.db")
conn.execute("CREATE TABLE IF NOT EXISTS alerts (entity TEXT, ts TEXT,
action TEXT)")
def already_alerted(entity, hours=24):
    cutoff = (datetime.datetime.now(datetime.timezone.utc) -
datetime.timedelta(hours=hours)).isoformat()
    return conn.execute("SELECT 1 FROM alerts WHERE entity=? AND ts>?
LIMIT 1", (entity, cutoff)).fetchone() is not None
# after a successful send: conn.execute("INSERT INTO alerts VALUES
(?,?,?)", (entity, now_iso, "email")); conn.commit()
```

BASH

```
crontab -e
# add:
*/30 * * * * cd $HOME/cloud-scripting/lab10 && $HOME/cloud-
scripting/lab01/.venv/bin/python alert_workflow.py >> alert.out 2>&1
```

Run the script twice by hand first: the second run must report that the finding was already alerted and send no second email.

Required Outcomes / Tasks

- Screenshot of the mail sink (Mailpit web UI or aiosmtpd terminal) showing one consolidated alert email, system clock visible.
- Both alert_workflow.py and alert_workflow.js submitted.
- Extend either script with your own list of 5 resources and a budget equal to the last two digits of your roll number; show the consolidated email.
- Output of `SELECT * FROM alerts;` after the state extension, and a second run that sends no email.

Lab 11: Error Handling and Debugging in Scripts

Course: CCSP-5213 | Scripting Program and Cloud Architecture | Week 12 | Program: BS Cloud Applications Development and Operations

Title

Error Handling and Debugging in Scripts

Objectives

- Apply try/except (Python) and try/catch (Node.js) to handle predictable cloud API failures.
- Use the free, built-in VS Code debugger to step through a script and inspect variables.
- Produce scripts that fail gracefully instead of crashing an automation pipeline.

Preamble / Short Theory

A cloud automation script that crashes on one malformed API response can halt an entire deployment pipeline. Defensive coding, catching specific exceptions, validating responses, retrying transient failures, is essential. VS Code's built-in debugger (free) allows breakpoints and live variable inspection at no cost, the same workflow used to debug production cloud automation code.

Prerequisites

- Lab 2 and Lab 9 completed.
- VS Code with the Python extension installed.
- Git installed and a remote on GitHub or Gitea (free) for the pull-request task.

Step-by-Step Guide

Step 1: Create the messy data file

Create messy_billing.csv with three deliberately malformed rows (a word, an empty value and a value with a stray letter).

csv

```
resource,cost
vm-01,72.5
db-02,abc
vm-03,
cache-04,19.99
vm-05,12x
```

Step 2: Run a deliberately fragile script

PYTHON

```
import csv

def parse_row(row):
    return {"resource": row["resource"], "cost": float(row["cost"])}
# crashes on non-numeric cost

with open("messy_billing.csv") as f:
    for row in csv.DictReader(f):
        print(parse_row(row))
```

Run it and read the traceback: it stops at the first bad row with a ValueError.

Step 3: Add targeted exceptions, timestamped logging and a custom exception

Catch only the exceptions you expect (KeyError, ValueError, TypeError), never a bare except. Log with a timestamp so the log can be placed on a timeline. RetryExhausted is a custom exception raised when every attempt of a retried operation fails. The retry helper catches OSError, because the exceptions of the requests library derive from OSError, not from the built-in ConnectionError.

PYTHON

```
import csv, logging, time

logging.basicConfig(filename="parse_errors.log",
level=logging.WARNING,
                    format="%(asctime)s %(levelname)s %(message)s")


class RetryExhausted(Exception):
    """Raised when every attempt of a retried operation has failed."""


def parse_row(row):
    try:
        return {"resource": row["resource"], "cost":
float(row["cost"])}
    except (KeyError, ValueError, TypeError) as exc:
        logging.warning("Skipped malformed row %r: %s", row, exc)
        return None


def with_retry(fn, attempts=3, delay=1.0, retry_on=(OSError,)):
    """Retry fn on transient errors. requests' exceptions subclass
OSError, NOT the builtin ConnectionError."""
    for i in range(1, attempts + 1):
        try:
            return fn()
        except retry_on as exc:
            logging.warning("Attempt %d/%d failed: %s", i, attempts,
exc)
            time.sleep(delay * 2 ** (i - 1))
    raise RetryExhausted(f"all {attempts} attempts failed")


if __name__ == "__main__":
    results = []
    try:
        with open("messy_billing.csv", newline="", encoding="utf-8")
as f:
            for row in csv.DictReader(f):
                parsed = parse_row(row)
                if parsed:
                    results.append(parsed)
    except FileNotFoundError:
        logging.error("messy_billing.csv not found")
        raise SystemExit("messy_billing.csv not found")
    print(f"Successfully parsed {len(results)} rows; see
parse_errors.log for skipped rows")
```

Expected: Successfully parsed 2 rows; see parse_errors.log for skipped rows. The log lines start with a date and time, for example 2026-10-09 10:53:38,467 WARNING Skipped malformed row ...

Step 4: Demonstrate the custom exception

Add this to the end of the script (inside the if __name__ block or in a separate file) and run it: every attempt fails, the retries are logged, and RetryExhausted is raised and caught.

PYTHON

```
def always_fails():
    raise OSError("simulated outage")

try:
    with_retry(always_fails, attempts=3, delay=0.1)
except RetryExhausted as exc:
    print("giving up:", exc)
```

Cloud track: prove why the retry helper must not catch the built-in ConnectionError, then retry a real request correctly.

BASH

```
python3 -c "import requests;
print(issubclass(requests.exceptions.ConnectionError,
ConnectionError))"   # False
```

PYTHON

```
import requests
with_retry(lambda: requests.get("http://127.0.0.1:1",
timeout=5).raise_for_status(),
           attempts=2, delay=0.1,
retry_on=(requests.exceptions.RequestException,))
```

Step 5: Debug with VS Code and with pdb

In VS Code, click to the left of the `return` line inside parse_row to set a breakpoint, press F5 (Run ▸ Start Debugging), open the Variables panel and step with F10. When the value of the bad field is visible, take your screenshot.

The same job in the terminal needs no editor: b parse_row sets a breakpoint, c continues, p row prints the row, n steps to the next line.

BASH

```
python3 -m pdb parse_messy.py
(Pdb) b parse_row
(Pdb) c
(Pdb) p row
(Pdb) n
(Pdb) c
```

Step 6: Node.js equivalent

In Node.js the same defensive pattern uses a check (or try/catch) and a timestamped warning:

JAVASCRIPT

```
function parseRow(row) {
  const cost = Number(row.cost);
  if (row.cost === '' || Number.isNaN(cost)) {
    console.warn(`${new Date().toISOString()} WARN skipped row:
invalid cost "${row.cost}"`);
    return null;
  }
  return { resource: row.resource, cost };
}
```

Add a few lines to run it over messy_billing.csv:

JAVASCRIPT

```
const fs = require('fs');
const [, ...lines] = fs.readFileSync('messy_billing.csv',
'utf8').trim().split('\n');
const rows = lines.map(l => { const [resource, cost] = l.split(',');
return { resource, cost }; });
const parsed = rows.map(parseRow).filter(Boolean);
console.log(`Successfully parsed ${parsed.length} rows`);
```

Step 7: Review your change through a pull request

Commit the corrected script on a branch, push it, open a pull request and ask a classmate to review it. Reviewers check that no bare except remains and that every skipped row is logged.

BASH

```
cd ~/cloud-scripting/lab11
git init
git add parse_messy.py messy_billing.csv
git commit -m "Lab 11: tolerant parser"
git switch -c feature/tolerant-parser
# push the branch and open the pull request on GitHub or Gitea
```

Required Outcomes / Tasks

- messy_billing.csv (with at least 3 deliberately malformed rows) and the corrected script submitted.
- parse_errors.log showing timestamped lines for the malformed rows being caught, not crashing the program.
- Screenshot of the VS Code debugger paused at a breakpoint with the Variables panel visible, or a pdb transcript showing `p row`, with the system clock visible.
- Screenshot of the RetryExhausted demonstration, and the link to your reviewed pull request.
- Two sentences: why is `except (KeyError, ValueError)` better than a bare `except`?

Lab 12: Logging and Monitoring in Cloud-Based Scripts

Course: CCSP-5213 | Scripting Program and Cloud Architecture | Week 13 | Program: BS Cloud Applications Development and Operations

Title

Logging and Monitoring in Cloud-Based Scripts

Objectives

- Replace ad-hoc print statements with Python's `logging` module and Node's `winston` (both free/OSS).
- Configure log levels, rotation, console output and structured (JSON) output.
- Make a log tamper-evident with a SHA-256 hash, and view and filter logs quickly using `lnav` (free, open-source log file navigator).

Preamble / Short Theory

Cloud operations rely on structured, filterable logs to diagnose failed deployments and cost anomalies across many automated jobs. Python's `logging` module and Node's `winston` package (both free/open source) provide leveled, rotating, structured logging suitable for feeding into log-aggregation systems later. `lnav` gives a free terminal UI for live filtering without needing a paid log-management SaaS.

Prerequisites

- Lab 11 completed.
- lnav installed (`sudo apt install lnav`).

Step-by-Step Guide

Step 1: Configure Python logging with rotation, JSON output and a console handler

One logger writes a text log, a JSON-lines log (one JSON object per line, easy for tools to read) and prints warnings and errors to the console. Both files rotate at 1 MB, keeping 3 old files.

BASH

```
cd ~/cloud-scripting && mkdir lab12 && cd lab12 &&
source ../lab01/.venv/bin/activate
```

PYTHON

```
import datetime, json, logging, sys
from logging.handlers import RotatingFileHandler


class JsonFormatter(logging.Formatter):
    def format(self, r):
        return json.dumps({"ts":
datetime.datetime.fromtimestamp(r.created,
datetime.timezone.utc).isoformat(),
                           "level": r.levelname, "msg":
r.getMessage(), "logger": r.name,
                           "context": getattr(r, "context", {})})


def make_logger(name="pipeline", max_bytes=1_000_000):
    logger = logging.getLogger(name)
    logger.setLevel(logging.DEBUG)
    text = RotatingFileHandler("pipeline.log", maxBytes=max_bytes,
backupCount=3, encoding="utf-8")
    text.setFormatter(logging.Formatter("%(asctime)s | %(levelname)s
| %(message)s"))
    jsonl = RotatingFileHandler("pipeline.jsonl", maxBytes=max_bytes,
backupCount=3, encoding="utf-8")
    jsonl.setFormatter(JsonFormatter())
    console = logging.StreamHandler(sys.stderr)
    console.setLevel(logging.WARNING)
    for h in (text, jsonl, console):
        logger.addHandler(h)
    return logger


if __name__ == "__main__":
    log = make_logger()
    log.info("Deployment pipeline started", extra={"context":
{"resource": "vm-01", "region": "us-east-1"}})
    log.warning("Resource vm-01 approaching budget limit")
    log.error("Failed to provision resource: quota exceeded")
```

Step 2: Run it and read the three outputs

BASH

```
python logger_demo.py
cat pipeline.log
cat pipeline.jsonl
```

Expected: the text log has three lines with a time stamp and level, the JSON log has the same events as JSON objects, and only the WARNING and ERROR messages appear on the console.

Step 3: Install and configure winston (Node.js)

BASH

```
npm install winston
```

Step 4: Write logger.js with built-in rotation

Winston's File transport already rotates with maxsize and maxFiles, so no extra package is needed (winston-daily-rotate-file is also acceptable if you prefer daily files).

JAVASCRIPT

```
const winston = require('winston');
const logger = winston.createLogger({
  level: 'debug',
  format: winston.format.combine(winston.format.timestamp(),
winston.format.json()),
  transports: [
    new winston.transports.File({ filename: 'pipeline.json.log',
maxsize: 1_000_000, maxFiles: 3 }),
    new winston.transports.Console(),
  ],
});
logger.info('Deployment pipeline started', { resource: 'vm-01',
region: 'us-east-1' });
logger.warn('Resource vm-01 approaching budget limit');
logger.error('Failed to provision resource: quota exceeded');
```

Step 5: Make a closed log tamper-evident

Lower max_bytes in the Python script to about 2000 and write 100 extra log lines in a loop so that the log rotates and a closed file such as pipeline.log.1 appears. Hash that closed file, store the hash somewhere else, and verify it later. The live pipeline.log changes with every line, so it must never be hashed while it is open.

BASH

```
# set maxBytes small (e.g. 2000) so the log rotates, then hash a
CLOSED file:
sha256sum pipeline.log.1 >> log_hashes.txt
# later, verify:
sha256sum -c log_hashes.txt        # OK, or FAILED if one character
was changed
```

Change one character in pipeline.log.1 and run the verification again: it must report FAILED.

Step 6: Inspect logs with lnav

BASH

```
lnav pipeline.log
# inside lnav: press e / E to jump to the next / previous error line,
# or type  :filter-in WARNING|ERROR  to show only those lines
```

Key bindings vary slightly between lnav versions; take the screenshot of the filtered view.

Required Outcomes / Tasks

- pipeline.log, pipeline.jsonl and pipeline.json.log submitted, with at least one message at each of the INFO, WARNING and ERROR levels.
- Screenshot of lnav showing a filtered view (only WARNING/ERROR lines), with the system clock visible.
- A rotated file (pipeline.log.1), its hash recorded in log_hashes.txt, and a screenshot of `sha256sum -c log_hashes.txt` showing OK, then FAILED after you change one character.
- Configure winston so that no single file exceeds 1 MB (maxsize and maxFiles, or winston-daily-rotate-file) and show the setting.
- Add the fields resource and region to the JSON context of every message.

Lab 13: Introduction to Docker: Containerizing Applications

Course: CCSP-5213 | Scripting Program and Cloud Architecture | Week 14 | Program: BS Cloud Applications Development and Operations

Title

Introduction to Docker: Containerizing Applications

Objectives

- Install Docker Engine Community Edition (Apache-2.0, free and open source).
- Write a Dockerfile that packages a cloud automation Python script and its dependencies.
- Build and run the container, confirming identical output to the host run.

Preamble / Short Theory

Containerization is foundational to cloud-native deployment: the same image runs identically on a laptop, in CI, and in production. Docker Engine CE is free and fully open source (Apache-2.0). This course standardizes on Docker Engine CE on Linux/WSL2 rather than Docker Desktop, avoiding Desktop's commercial-use licensing restrictions; Podman is offered as a fully open-source, daemon-less alternative for students who prefer it.

One rule prevents the most common failure in this lab: a bind mount (-v) replaces whatever the image has at that path with the host folder. Keep the code in /app and mount only a separate output folder (/app/out). The container also runs as a non-root user, which limits the damage if the program is ever compromised.

Prerequisites

- Lab 1, Step 7 completed (Docker installed and `docker run --rm hello-world` works).
- A working corrected script from Lab 3 (and sample_data.csv from Lab 2).

Step-by-Step Guide

Step 1: Confirm Docker Engine CE

BASH

```
docker --version
docker run --rm hello-world
# Fully open-source daemon-less alternative: sudo apt install podman
```

Step 2: Prepare the application

BASH

```
mkdir -p ~/cloud-scripting/lab13 && cd ~/cloud-scripting/lab13
cp ../lab03/log_filter_cli.py .
cp ../lab02/sample_data.csv .
```

Step 3: Write the Dockerfile and .dockerignore

The image is small, creates a non-root user, and keeps the output folder separate from the code. No requirements.txt is needed because the script uses only the standard library. Use python:3.13-slim if that is what your lab PC provides; the version is not important for the exercise.

DOCKERFILE

```
FROM python:3.12-slim
WORKDIR /app
COPY log_filter_cli.py sample_data.csv ./
RUN useradd --create-home appuser && mkdir -p /app/out && chown
appuser /app/out
USER appuser
ENTRYPOINT ["python", "log_filter_cli.py"]
CMD ["--input", "sample_data.csv", "--output", "/app/out/output.csv",
"--level", "WARN"]
```

Add a .dockerignore file so that the virtual environment, Git data, secrets, logs and earlier output never enter the image:

TEXT

```
.venv/
.git/
__pycache__/
.env
*.log
out/
```

Step 4: Build the image

BASH

```
docker build -t cloud-log-filter:1.0 .
```

Step 5: Run the container and retrieve the output

Mount only the out sub-folder. Mounting a host folder over /app would hide log_filter_cli.py and the run would fail with “can't open file”.

BASH

```
docker build -t cloud-log-filter:1.0 .
mkdir -p out
docker run --rm --user "$(id -u):$(id -g)" -v "$PWD/out:/app/out"
cloud-log-filter:1.0
cat out/output.csv | head
# PowerShell: -v "${PWD}/out:/app/out" (and omit --user)
docker images cloud-log-filter
docker history cloud-log-filter:1.0
```

Expected: the console prints N rows with level=WARN written to /app/out/output.csv (0 skipped), and out/output.csv appears on the host. Compare it with a host run; the two hashes must match:

BASH

```
python log_filter_cli.py --input sample_data.csv --output
host_output.csv --level WARN
sha256sum out/output.csv host_output.csv
```

Step 6: Inspect the image

BASH

```
docker images cloud-log-filter
docker history cloud-log-filter:1.0
```

Step 7: Prove the build cache, .dockerignore and --env-file

Build twice without changing anything: every layer is reused (shown as CACHED). Then change only log_filter_cli.py and rebuild: layers before the COPY line stay cached, which is why files that change often are copied late. Finally pass a setting with --env-file.

BASH

```
docker build -t cloud-log-filter:1.0 .        # run it a second time:
layers show CACHED
printf 'LEVEL=WARN\n' > .env
docker run --rm --env-file .env --entrypoint sh cloud-log-filter:1.0 -
c 'echo "LEVEL is $LEVEL"'
```

To prove that .dockerignore works, create a file named secret.env, rebuild, and confirm that `docker run --rm --entrypoint ls cloud-log-filter:1.0 /app` does not list it (add secret.env to .dockerignore first).

Required Outcomes / Tasks

- Dockerfile and .dockerignore submitted along with `docker images` output showing the built image (system clock visible).
- out/output.csv produced by the containerized run, with `sha256sum` showing that it matches the host-run output from Lab 3.
- Screenshot of the second build showing CACHED layers, and of the --env-file demonstration.
- One paragraph: what would need to change in the Dockerfile to add a healthcheck for this container, and does a run-once batch job need one?

Lab 14: Automating Container Deployment

Course: CCSP-5213 | Scripting Program and Cloud Architecture | Weeks 14–15 | Program: BS Cloud Applications Development and Operations

Title

Automating Container Deployment

Objectives

- Run a free, self-hosted Docker Registry (open source) instead of a paid Docker Hub plan.
- Run Moto (free, open-source S3 emulator) to simulate cloud object storage without an AWS account.
- Tag, push, and pull the Lab 13 image, and upload and download an artefact to and from the emulated bucket.

Preamble / Short Theory

This lab pairs two free, open-source stand-ins for paid cloud services: the official `registry:2` image (Apache-2.0) for container images, and Moto, an S3 emulator that speaks the same API as AWS S3, so any `boto3`/AWS-SDK code written against it also works against real S3 later, with zero cost during coursework. (MinIO, used in earlier editions, no longer publishes community images and its repository is in maintenance mode.) The lab registry has no TLS and no authentication, so use it only on your own machine or a closed lab network.

Prerequisites

- Lab 13 completed (cloud-log-filter:1.0 image built).
- Docker running.

Step-by-Step Guide

Step 1: Run a local open-source registry

BASH

```
docker run -d -p 5000:5000 --restart=always --name local-registry
registry:2
```

Step 2: Tag and push the image

BASH

```
docker tag cloud-log-filter:1.0 localhost:5000/cloud-log-filter:1.0
docker push localhost:5000/cloud-log-filter:1.0
```

Step 3: Verify the push

BASH

```
curl http://localhost:5000/v2/_catalog
#   {"repositories":["cloud-log-filter"]}
curl http://localhost:5000/v2/cloud-log-filter/tags/list
#   {"name":"cloud-log-filter","tags":["1.0"]}
```

Step 4: Make sure the artefact exists

Lab 13 should have produced out/output.csv. If it is missing, run the container again.

BASH

```
cd ~/cloud-scripting/lab13
ls out/output.csv || docker run --rm --user "$(id -u):$(id -g)" -v
"$PWD/out:/app/out" cloud-log-filter:1.0
```

Step 5: Run Moto and upload and download the artefact

Start the emulator, upload out/output.csv to a bucket, list the bucket, then download the file again and compare it with the original.

BASH

```
pip install "moto[server]" boto3
moto_server -p 5000 &
python3 - <<'PY'
import boto3
s3 = boto3.client("s3", endpoint_url="http://localhost:5000",
region_name="us-east-1",
                  aws_access_key_id="test",
aws_secret_access_key="test")
s3.create_bucket(Bucket="pipeline-artifacts")
s3.upload_file("out/output.csv", "pipeline-artifacts", "output.csv")
print([o["Key"] for o in s3.list_objects_v2(Bucket="pipeline-
artifacts")["Contents"]])
s3.download_file("pipeline-artifacts", "output.csv", "downloaded.csv")
# the step the manual never does
PY
cmp out/output.csv downloaded.csv && echo "identical"
```

Step 6: Simulate a second host: pull the image and run it

BASH

```
docker rmi cloud-log-filter:1.0 localhost:5000/cloud-log-filter:1.0
docker pull localhost:5000/cloud-log-filter:1.0
mkdir -p out2
docker run --rm --user "$(id -u):$(id -g)" -v "$PWD/out2:/app/out"
localhost:5000/cloud-log-filter:1.0
```

Required Outcomes / Tasks

- Output of `curl http://localhost:5000/v2/_catalog` showing the pushed image (system clock visible).
- The list_objects_v2 output showing output.csv in the pipeline-artifacts bucket, and `identical` from the cmp command.
- Screenshot of the pull-then-run sequence proving the image was retrieved fresh from the registry.
- Short note comparing a self-hosted S3 emulator with a managed cloud storage free tier (durability, scaling, IAM, limits, account and billing).

Lab 15: CI/CD Pipeline Automation

Course: CCSP-5213 | Scripting Program and Cloud Architecture | Week 15 | Program: BS Cloud Applications Development and Operations

Title

CI/CD Pipeline Automation

Objectives

- Build a CI/CD pipeline that builds, tests, scans for vulnerabilities and pushes the tool image automatically on every commit.
- Route A: run Jenkins (free, open source, self-hosted) with a Jenkinsfile that polls the repository.
- Route B: write the equivalent GitHub Actions workflow, which is the route used in the lecture notes and in Assignment 3.

Preamble / Short Theory

CI/CD automates build-test-deploy so every change is validated consistently, and a vulnerability scan (Trivy) fails the build when serious problems are found. Jenkins is free, open source (MIT) and self-hostable with no per-seat costs, a common alternative to SaaS-only CI/CD in cost-sensitive or air-gapped cloud environments; GitHub Actions has a free tier and is what the course notes and Assignment 3 use. Complete Route A (Jenkins) and then Route B, which takes about 15 minutes. Both wire the pipeline to the Git repository and local registry built in Labs 13–14.

Prerequisites

- Labs 13–14 completed (registry running on localhost:5000).
- Git installed; the lab13 folder will become the repository.
- Docker running (Jenkins runs as a container). Route B needs a free GitHub account.

Step-by-Step Guide

Step 1: Route A: initialise the Git repository

The Jenkinsfile belongs in the lab13 repository, next to the Dockerfile.

BASH

```
cd ~/cloud-scripting/lab13
git init
git add Dockerfile .dockerignore log_filter_cli.py sample_data.csv
git commit -m 'Initial tool'
```

Step 2: Write the Jenkinsfile (build, test, scan, push)

The pipeline polls the repository every two minutes, builds the image, runs it as a test, scans it with Trivy and only then pushes. Test comes first so a broken image is never pushed; `--exit-code 1` makes the scan stage fail the build when HIGH or CRITICAL vulnerabilities with a fix are found. Replace <PINNED_VERSION> with the Trivy release your instructor announces (pinning a version avoids surprise changes).

GROOVY

```
pipeline {
  agent any
  triggers { pollSCM('H/2 * * * *') }
  environment {
    IMAGE = "localhost:5000/cloud-log-filter"
    TAG = "${env.BUILD_NUMBER}"
  }
  stages {
    stage('Build') { steps { sh 'docker build -t $IMAGE:$TAG .' } }
    stage('Test') {
      steps { sh 'docker run --rm $IMAGE:$TAG --input sample_data.csv
--output /tmp/out.csv --level WARN' }
    }
    stage('Scan') {
      steps {
        sh 'docker run --rm -v
/var/run/docker.sock:/var/run/docker.sock
aquasec/trivy:<PINNED_VERSION> image --exit-code 1 --severity
HIGH,CRITICAL --ignore-unfixed $IMAGE:$TAG'
      }
    }
    stage('Push') { steps { sh 'docker push $IMAGE:$TAG' } }
  }
  post {
    success { echo 'Pipeline completed: image built, tested, scanned
and pushed.' }
    failure { echo 'Pipeline failed: check the console output.' }
  }
}
```

BASH

```
cd ~/cloud-scripting/lab13
git add Jenkinsfile && git commit -m 'Add Jenkinsfile'
```

Step 3: Build a Jenkins image that contains the Docker CLI

The official Jenkins image has no Docker CLI, so docker build would fail with “docker: not found”. Build a small image on the lts-jdk21 tag (Jenkins now requires Java 21).

DOCKERFILE

```
# Dockerfile.jenkins
FROM jenkins/jenkins:lts-jdk21
USER root
COPY --from=docker:cli /usr/local/bin/docker /usr/local/bin/docker
USER jenkins
```

BASH

```
docker build -f Dockerfile.jenkins -t jenkins-docker:lts .
```

Step 4: Run Jenkins

The container needs three things: the Docker socket (with its group), the repository as a read-only folder it can see, and a volume for its own data.

BASH

```
docker build -f Dockerfile.jenkins -t jenkins-docker:lts .
docker volume create jenkins_home
docker run -d --name jenkins -p 8080:8080 -p 50000:50000 \
  -v jenkins_home:/var/jenkins_home \
  -v /var/run/docker.sock:/var/run/docker.sock \
  --group-add "$(stat -c %g /var/run/docker.sock)" \
  -v "$HOME/cloud-scripting/lab13:/repo:ro" \
  jenkins-docker:lts
docker exec jenkins git config --global --add safe.directory /repo
docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword
```

Step 5: Complete setup in the browser

Open http://localhost:8080, paste the initial admin password, choose “Install suggested plugins” (all free/OSS), and create an admin user.

Step 6: Create the pipeline job and run it

New Item ▸ Pipeline ▸ name it cloud-log-filter-pipeline ▸ under Pipeline choose “Pipeline script from SCM” ▸ SCM: Git ▸ Repository URL /repo (or a GitHub/Gitea URL) ▸ Script Path Jenkinsfile. Choose Build Now, then make a small change, commit it, and wait for the poll to start a build by itself.

BASH

```
# trigger from the Jenkins UI: 'Build Now', then prove the commit
trigger:
cd ~/cloud-scripting/lab13
echo '# change' >> log_filter_cli.py && git commit -am 'Trigger build'
curl http://localhost:5000/v2/cloud-log-filter/tags/list     # the new
build number appears
```

Step 7: Route B: the same pipeline in GitHub Actions

Create .github/workflows/ci.yml in your repository, push to GitHub, and open the Actions tab. Permissions are limited to reading the repository, and the Trivy action should be pinned to a released tag or commit that you have checked; replace <PINNED_TAG_OR_SHA> and check the current major versions of actions/checkout and actions/setup-python.

YAML

```yaml
name: CI
on:
  push:
  pull_request:
permissions:
  contents: read
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - run: python -m py_compile log_filter_cli.py
      - run: python log_filter_cli.py --input sample_data.csv --output
/tmp/out.csv --level WARN
  build-and-scan:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: docker build -t cloud-log-filter:${{ github.sha }} .
      - uses: aquasecurity/trivy-action@<PINNED_TAG_OR_SHA>
        with:
          image-ref: cloud-log-filter:${{ github.sha }}
          severity: CRITICAL,HIGH
          ignore-unfixed: true
          exit-code: "1"
```

Step 8: Protect the main branch

In the repository settings add a branch protection rule for main: require a pull request and require the CI checks to pass before merging. Then open a pull request that breaks the build on purpose and show that it cannot be merged.

Required Outcomes / Tasks

- Jenkinsfile submitted alongside a screenshot of a green pipeline run showing the Build, Test, Scan and Push stages, with the system clock visible.
- Console output log from the Test stage showing the containerised script executed correctly.
- Confirm via `curl http://localhost:5000/v2/cloud-log-filter/tags/list` that the new build tag was pushed.
- Route B: ci.yml submitted with a screenshot of a green GitHub Actions run, and a screenshot of the branch-protection rule.
- Short answer: why does Test come before Build/Push, and what does `exit-code: 1` do in the scan stage?

Lab 16: Capstone Lab: Designing and Deploying a Complete Cloud-Native Application

Course: CCSP-5213 | Scripting Program and Cloud Architecture | Week 16 | Program: BS Cloud Applications Development and Operations

Title

Capstone Lab: Designing and Deploying a Complete Cloud-Native Application

Objectives

- Integrate scripting, error handling, logging, containerization, storage, and CI/CD from Labs 1–15 into one pipeline.
- Design a pipeline that ingests usage data, checks budgets, stores artefacts in MinIO, alerts via email, and deploys automatically.
- Present and defend design decisions in a short demo to the class/instructor.

Preamble / Short Theory

The capstone consolidates the semester's tools into one cloud-native pipeline end to end: a Bash orchestration script (Lab 5) runs a Python filtering script (Labs 2–3) with timestamped logging (Lab 12) and defensive error handling (Lab 11); each filtered usage row is converted to JSON and sent to the usage-alert function (Lab 8, through the local harness or faasd); the answers are read by an alert script that e-mails through the SMTP sink (Lab 10); the results are archived with a SHA-256 hash and uploaded to the S3 emulator (Lab 14); and the whole toolchain is containerised (Lab 13), pushed to the local registry (Lab 14), and built automatically by CI/CD (Lab 15). Every stage reads exactly what the previous stage wrote, and all components remain free and open source.

Prerequisites

- Labs 1–15 completed and their artifacts available.
- The SMTP sink from Lab 10 running (Mailpit or aiosmtpd on port 1025) and the Moto server from Lab 14 running (optional, for the upload stage).
- A short (5–10 minute) in-class or recorded demo slot booked with the instructor.

Step-by-Step Guide

Step 1: Design the pipeline

Sketch a diagram (hand-drawn, draw.io free tier, or the OSS tool Excalidraw) showing data flow: usage data → filter script → budget check (JSON per row) → alert → archive and upload → CI/CD. Submit the diagram alongside the code.

Step 2: Assemble the project repository

Copy the files from earlier labs. The Jenkinsfile and Dockerfile come from lab13 (that is where Lab 15 created them).

BASH

```
mkdir -p ~/cloud-scripting/capstone && cd ~/cloud-scripting/capstone
git init
mkdir -p data scripts faas docker jenkins .github/workflows out
cp ../lab03/log_filter_cli.py scripts/
cp ../lab08/handler.js ../lab08/harness.js faas/
cp ../lab13/Dockerfile docker/Dockerfile.lab13
cp ../lab13/Jenkinsfile jenkins/
cp ../lab13/.github/workflows/ci.yml .github/workflows/ 2>/dev/null ||
true   # only if you completed Route B
```

Target layout:

TEXT

```
capstone/
  data/raw.csv
  scripts/log_filter_cli.py     (Lab 3)
  scripts/alert_from_scan.py
  faas/handler.js  faas/harness.js   (Lab 8)
  docker/Dockerfile   jenkins/Jenkinsfile   .github/workflows/ci.yml
  run_pipeline.sh   diagram.png   README.md
```

Step 3: Supply the input data

The pipeline needs data/usage.csv, including one deliberately invalid row.

CSV

```
resource,costUsd,level
vm-01,72.5,WARN
db-02,20.0,INFO
vm-03,130.0,WARN
bad-row,abc,WARN
```

Step 4: Write the budget-check script

The earlier draft posted a CSV to a function that calls JSON.parse, which throws. This script converts each row to JSON and calls the function once per row; an invalid row is rejected but does not stop the run. With SCAN_URL unset it uses the local harness.

PYTHON

```
"""Send each filtered usage row to the usage-alert function and
collect the answers."""
import csv, json, os, subprocess, sys, urllib.request

src, dst = sys.argv[1], sys.argv[2]
url = os.getenv("SCAN_URL")                          # e.g.
http://host.docker.internal:8080/function/usage-alert
results = []
with open(src, newline="", encoding="utf-8") as f:
    for row in csv.DictReader(f):
        payload = json.dumps({"resource": row["resource"], "costUsd":
row["costUsd"]})
        if url:
            req = urllib.request.Request(url, data=payload.encode(),
headers={"Content-Type": "application/json"})
            with urllib.request.urlopen(req, timeout=10) as r:
                results.append(json.load(r))
        else:
            out = subprocess.run(["node", "faas/harness.js",
"faas/handler.js", payload], capture_output=True, text=True,
check=True)
            results.append(json.loads(out.stdout)["body"])
with open(dst, "w", encoding="utf-8") as f:
    json.dump(results, f, indent=2)
errors = [r for r in results if "error" in r]
over = [r for r in results if r.get("overBudget")]
print(f"{len(results) - len(errors)} resources checked, {len(errors)}
rejected as invalid, {len(over)} over budget: {[r['resource'] for r in
over]}")
```

Step 5: Write the alert and upload scripts

alert_from_scan.py reads the budget-check result and e-mails the resources over budget. upload_to_s3.py sends the archive to the S3 emulator.

PYTHON

```
import json, os, smtplib, sys
from email.mime.text import MIMEText

with open(sys.argv[1], encoding="utf-8") as f:
    results = json.load(f)
over = [r for r in results if r.get("overBudget")]
if not over:
    print("Within budget, no alert sent")
    sys.exit(0)

msg = MIMEText("Resources over budget:\n" + "\n".join(f"-
{r['resource']}: ${r['costUsd']:.2f}" for r in over))
msg["Subject"] = f"Cloud Cost Alert: {len(over)} resource(s) over
budget"
msg["From"], msg["To"] = "cost-automation@lab.local", "ops@lab.local"
try:
    with smtplib.SMTP(os.getenv("MAIL_HOST", "localhost"),
int(os.getenv("MAIL_PORT", "1025")), timeout=10) as s:
        s.send_message(msg)
except (OSError, smtplib.SMTPException) as exc:
    sys.exit(f"Alert failed: {exc}")
print("Alert sent for:", [r["resource"] for r in over])
```

PYTHON

```
import os, sys, boto3
s3 = boto3.client("s3", endpoint_url=os.getenv("S3_URL",
"http://localhost:5000"), region_name="us-east-1",
                  aws_access_key_id=os.getenv("AWS_ACCESS_KEY_ID",
"test"),

aws_secret_access_key=os.getenv("AWS_SECRET_ACCESS_KEY", "test"))
bucket, path = "pipeline-artifacts", sys.argv[1]
try:
    s3.create_bucket(Bucket=bucket)
except s3.exceptions.BucketAlreadyOwnedByYou:
    pass
s3.upload_file(path, bucket, os.path.basename(path))
print("uploaded", path, "to", bucket)
```

Step 6: Write the orchestration script and run it

BASH

```
#!/usr/bin/env bash
# Cloud capstone orchestrator: stops at the first failure; logs each
stage with a UTC timestamp.
set -euo pipefail
cd "$(dirname "$0")"
mkdir -p data out
log() { echo "$(date -u +%FT%TZ) $*" | tee -a out/pipeline.log; }

log "[1/5] Filtering usage data"
python3 scripts/log_filter_cli.py --input data/usage.csv --output
data/filtered.csv --level WARN
log "[2/5] Checking budgets"
python3 scripts/check_budgets.py data/filtered.csv
data/scan_result.json
log "[3/5] Alerting if over budget"
python3 scripts/alert_from_scan.py data/scan_result.json
ARCHIVE="out/archive_$(date -u +%Y%m%d_%H%M%S).tar.gz"
log "[4/5] Archiving"
tar -czf "$ARCHIVE" data/
sha256sum "$ARCHIVE" >> out/SHA256SUMS
if [ -n "${S3_URL:-}" ]; then                          # skipped when
no S3 endpoint is configured
  log "[5/5] Uploading to the S3 endpoint"
  python3 scripts/upload_to_s3.py "$ARCHIVE"
fi
log "Pipeline complete: $ARCHIVE"
```

BASH

```
chmod +x run_pipeline.sh
THRESHOLD_USD=50 ./run_pipeline.sh                          # upload
stage skipped
S3_URL=http://localhost:5000 THRESHOLD_USD=50 ./run_pipeline.sh   #
with Moto running
sha256sum -c out/SHA256SUMS
```

Expected: 3 rows are filtered, 2 resources are checked and 1 is rejected as invalid, vm-01 and vm-03 are over a 50 threshold, an alert e-mail is sent, and the archive appears in the bucket listing when S3_URL is set.

Step 7: Containerise the pipeline

Inside a container 127.0.0.1 means the container itself, so the function and the mail server on the host are reached through host.docker.internal. The image needs boto3 for the upload stage and runs as a non-root user. Only the out folder is mounted.

DOCKERFILE

```
FROM python:3.12-slim
RUN pip install --no-cache-dir boto3
WORKDIR /app
COPY . .
RUN useradd --create-home appuser && mkdir -p data out && chown -R
appuser /app
USER appuser
ENTRYPOINT ["./run_pipeline.sh"]
```

BASH

```
docker build -f docker/Dockerfile -t cloud-pipeline:1.0 .
docker run --rm --add-host=host.docker.internal:host-gateway \
  -e SCAN_URL=http://host.docker.internal:8080/function/usage-alert \
  -e MAIL_HOST=host.docker.internal \
  -e S3_URL=http://host.docker.internal:5000 \
  -v "$PWD/out:/app/out" cloud-pipeline:1.0
```

Push the image to the local registry as cloud-pipeline:1.0 (docker tag, then docker push localhost:5000/cloud-pipeline:1.0).

Step 8: Wire it into CI/CD

Reuse the Lab 15 pipeline with an added Deploy stage (Jenkins) or job (GitHub Actions) that builds the new pipeline image and runs it on every successful build.

Step 9: Prepare the demo

Be ready to show: the pipeline diagram, a live or recorded run, the CI/CD build history, the alert e-mail in the SMTP sink, the archive in the S3 bucket listing, and your answer to “what happens if the function is down?”.

Required Outcomes / Tasks

- Complete capstone repository (scripts/, faas/, docker/, jenkins/, run_pipeline.sh, and the design diagram).
- A successful end-to-end run log (or screen recording, system clock visible) from ingestion through to the archived output, its verified SHA-256, and the alert email.
- 5–10 minute live or recorded demo presented to the instructor, with a short Q&A on design trade-offs.
- One-page written reflection: which lab's tool was hardest to integrate, and how was the problem resolved?

Change Log: Fall 2026 Revision

This edition was regenerated from the Fall 2026 verification report (35 findings: 11 blockers, 12 major, 12 minor) using the corrected code of the solution pack. The table lists every finding and what was changed in this manual.

Changes made for each finding

#	Lab(s)	Severity	Change made
1	DFSP 1–2, CCSP 1–2	Blocker	Python requirement changed from “3.13 or later” to “3.12 or newer” in the steps, tool table, prerequisites and the screenshot task.
2	DFSP 3, CCSP 3	Blocker	log_filter_cli.py now counts written rows in a plain loop, handles a missing column, reports missing input with exit code 1, and gains --limit and timestamped logging.
3	DFSP 4, CCSP 4	Blocker	organize.sh skips its own file name, so it no longer moves itself and a second run works.
4	DFSP 4, CCSP 4	Major	Summary counts per category (logs, csv, other); name clashes keep both copies by prefixing the time.
5	DFSP 5, CCSP 5	Major	backup.sh excludes .venv and node_modules, writes and verifies SHA-256, applies 7-day retention; script path corrected to ~/<project>/lab05; CCSP Lab 5 gains the Moto + aws CLI step.
6	DFSP 7, CCSP 7	Minor	Renamer is a dry run unless --apply is given, never overwrites, exits with code 1 on error; Node “22 or 24”.
7	DFSP 8, CCSP 8	Blocker	Lab 8 split into Tier 1 (local harness, graded) and Tier 2 (faasd); Tier 2 now starts registry:2 and uses a 127.0.0.1:5000 image name.
8	DFSP 8, CCSP 8	Major	Removed the claim that OpenFaaS/faasd is Apache-2.0 and fully open source; faasd CE EULA and the systemd requirement are stated; the harness works for every student.
9	DFSP 8, CCSP 8	Minor	Template choice no longer fixed to node20 (end of life); faas-cli installer is downloaded and read before running; Docker comes from apt (Lab 1).
10	DFSP 9, CCSP 9	Major	Evidence workflow: status check, timestamped evidence file, SHA-256, custody log, .env key and 429 handling task, sqlite3 installed; local mock API added as fallback.
11	CCSP 9–10	Major	Moto emulator step added to Lab 9 (cloud-state JSON saved as evidence). Not done: Lab 10 still uses a fixed five-resource list rather than reading costs from the emulator (see Open items).
12	DFSP 10, CCSP 10	Major	MailHog replaced by Mailpit or aiosmtpd; SQLite state and a cron schedule added; CCSP theory no longer says Lab 8 logic is called.

#	Lab(s)	Severity	Change made
13	DFSP 11, CCSP 11	Major	messy_log.csv / messy_billing.csv supplied; timestamped log format; custom exception, pdb and a Git pull-request task added.
14	CCSP 11	Blocker	Retry helper catches OSError / requests.exceptions.RequestException instead of the built-in ConnectionError, with a one-line proof.
15	DFSP 12, CCSP 12	Minor	JSON handler, console handler and a hash-of-closed-log tamper-evidence step added; lnav filter keys corrected.
16	DFSP 13, CCSP 13	Blocker	Bind mount changed to -v "$PWD/out:/app/out" with output in /app/out; non-root user, .dockerignore added; pointless pip upgrade removed.
17	DFSP 14, CCSP 14	Blocker	Second-host run mounts out2:/app/out; CCSP Lab 14 first checks that out/output.csv exists.
18	CCSP 14	Blocker	MinIO replaced by the Moto S3 emulator; console screenshot replaced by list_objects_v2 output; download and cmp step added.
19	DFSP 14	Minor	Docker Hub wording changed to pull-rate limits and a limited number of private repositories (check current terms).
20	DFSP 15, CCSP 15	Blocker	Jenkins image built with the Docker CLI; socket group added; lab13 repository mounted at /repo; pollSCM trigger added.
21	DFSP 15, CCSP 15	Major	Jenkins tag changed to lts-jdk21; Trivy scan stage added; GitHub Actions route added.
22	DFSP 16	Blocker	Jenkinsfile copied from lab13; raw.csv supplied; alert_from_scan.py reads the scan result; SCAN_URL and host.docker.internal replace 127.0.0.1; curl installed in the image.
23	CCSP 16	Blocker	Each row is converted to JSON and sent separately; upload_to_s3.py provided; budgets come from the data, not a hard-coded demo.
24	All	Major	Lab week labels corrected (Labs 9–16 follow the midterm) and a mapping table added to How to Use This Manual.
25	All	Major	Windows students are told to use WSL2 with systemd; PowerShell differences noted for venv activation and Docker mounts.
26	All	Major	curl | sh installs removed: Docker via apt, Node.js via snap or a downloaded and inspected script, faas-cli downloaded and read.
27	All	Minor	UTC timestamps and random.seed(42) in the generators and scripts.

#	Lab(s)	Severity	Change made
28	All	Minor	“Latest stable” wording replaced by minimum versions in the cover note, tool table and labs.
29	DFSP 6, CCSP 6	Minor	Pause moved between requests; column -s, removed; local test page added; feed-first guidance added.
30	CCSP 2–3, 6–7	Major	Partly done: resources.csv generator added to CCSP Lab 2 and cloud wording kept; Labs 3, 6 and 7 still share code with Part I (see Open items).
31	Both	Minor	Every outcomes list asks for the system clock in screenshots; personal values added (roll number in Lab 1, budget in CCSP Lab 10).
32	DFSP 12	Minor	Winston built-in maxsize/maxFiles accepted as well as winston-daily-rotate-file.
33	Both 8	Minor	Docker is installed once, in Lab 1 Step 7; later labs only confirm it.
34	Both	Minor	Git tasks added in Labs 1, 3, 7, 11 and 15.
35	Both 15–16	Minor	Both CI routes are in Lab 15 and the choice is stated: Route A Jenkins, then Route B GitHub Actions (needed for Assignment 3).

New material added

- Lab 1 Step 7 (Git and Docker once) and Step 8 (.gitignore and first commit).
- Lab 2: exception primer; CCSP resources.csv generator.
- Lab 6: local test page for offline testing; Lab 9: local mock API.
- Lab 12: tamper-evidence step; Lab 13: build-cache, .dockerignore and --env-file demonstrations.
- Lab 15: Trivy scan and GitHub Actions route; Lab 16: complete connected pipelines for both courses.
- Evidence rules, Windows/WSL2 guidance and the lab-to-week mapping in How to Use This Manual.

Formatting standard applied

Element	Setting
Regular text	Aptos 12, justified
Headings	Aptos bold in deep dark purple: 18 (parts, lab titles, major sections), 16 (section headings), 14 (step titles); justified
Code and configuration	Aptos Mono 12, left aligned, in a grey box with a purple left bar and a language label
Tables	Aptos 12, justified, deep dark purple header row with white text
Page	A4 portrait, margins, header and footer unchanged; each lab starts on a new page

Verification status and open items

- Code in Labs 1–7 and 9–12 comes from the tested solution pack, and the new snippets (mock API, resources generator, Lab 8 harness and handlers, CCSP alert, parsers, both capstone pipelines against a local SMTP sink) were run when this edition was built. Not run: faasd (Lab 8 Tier 2), Docker (Labs 13, 14, 16 container steps), Jenkins and GitHub Actions (Lab 15), Moto/aws CLI steps and the public websites; please run Labs 13 and 15 once on a lab PC before term.
- Decision needed: whether the faasd CE EULA covers classroom use. Until then Tier 1 of Lab 8 is the graded route.
- Placeholders to fill before term: the Trivy version in the Jenkinsfile and the pinned Trivy action tag in ci.yml; check the current major versions of the GitHub actions used.
- Open: CCSP Lab 10 does not yet read costs from the Moto emulator; CCSP Labs 3, 6 and 7 still share code with Part I.
- Re-check the Moto cloud-state path (/moto-api/data.json) on the version you install.